APPLIED SCIENTIST

Complete Interview Preparation Guide

Microsoft & Amazon — Expanded Edition

PART 1: Mathematical Foundations (Linear Algebra, Calculus, Optimization)
PART 2: Probability & Statistics (Deep Dive + Causal Inference)
PART 3: ML Algorithms (Full Mathematical Derivations)
PART 4: Deep Learning (CNN, RNN, Transformer, LLM Fine-tuning, LoRA)
PART 5: NLP (Tokenization, BERT, GPT, RAG, Embeddings)
PART 6: ML System Design (Recommendations, Fraud, Search, MLOps)
PART 7: Coding & Algorithms (ML from Scratch + LeetCode Patterns)
PART 8: Advanced Topics (GNN, RL, Optimization, Fairness, Distributed Training)
PART 9: Top 50 Deep Q&A (Scenario-based, Derivation, System Design)
PART 10: Behavioral & Leadership Principles

PART 1: MATHEMATICAL FOUNDATIONS FOR ML

1. Linear Algebra for Machine Learning

Linear algebra is the language of ML. Weights are vectors, datasets are matrices, transformations are linear maps. You WILL be asked to derive things on a whiteboard.

1.1 Vectors and Matrices — Core Operations

Vector	An ordered list of numbers. In ML: a data point is a vector of features. Shape: (d,) or (d,1). Operations: dot product, norm, distance.
Matrix	A 2D array of numbers. In ML: a dataset of N samples with d features = matrix of shape (N, d). A weight matrix maps from one space to another.
Transpose	A^T flips rows and columns. (AB)^T = B^T A^T. Critical identity: the gradient of x^T A x with respect to x is (A + A^T)x.
Matrix Multiplication	C = AB where C_ij = sum_k A_ik * B_kj. Dimensions: (m,k)*(k,n) -> (m,n). The inner dimensions must match.

```
Linear Regression: y = Xw + b,  solved analytically as w = (X^T X)^-1 X^T y
```

This is the Normal Equation. X^T X must be invertible (full rank). Computationally expensive for large N; gradient descent is preferred.

Eigenvalues and Eigenvectors

For square matrix A: Av = λv. v is the eigenvector; λ is the eigenvalue. Eigenvectors show the directions that A only stretches or shrinks (doesn't rotate). PCA uses eigendecomposition of the covariance matrix.

```
Covariance Matrix: C = (1/N) X^T X   (after centering X)
```

Eigenvectors of C = principal components. Eigenvalues = variance explained. PCA: project X onto top-k eigenvectors.

SVD — Singular Value Decomposition

Any matrix A = U Σ V^T. U = left singular vectors (input directions), Σ = diagonal singular values, V^T = right singular vectors (output directions).

- SVD generalizes eigendecomposition to non-square matrices
- PCA via SVD: more numerically stable than eigendecomposition of X^T X
- Low-rank approximation: keep top-k singular values to compress data (used in recommender systems, image compression)
- Condition number = σ_max / σ_min. Large condition number = ill-conditioned matrix = gradient descent will oscillate

```
# SVD and PCA
import numpy as np

A = np.random.randn(100, 50)
U, S, Vt = np.linalg.svd(A, full_matrices=False)
# A approx U[:, :k] @ np.diag(S[:k]) @ Vt[:k, :]  -- rank-k approximation

# PCA via SVD (sklearn does this internally)
X_centered = A - A.mean(axis=0)
U, S, Vt = np.linalg.svd(X_centered, full_matrices=False)
X_pca = X_centered @ Vt[:10].T   # project to 10 dims
```

Norms — Measuring Vector Size

L1 Norm	||x||_1 = sum |x_i|. Sum of absolute values. Induces sparsity when used as regularizer (Lasso).
L2 Norm	||x||_2 = sqrt(sum x_i^2). Euclidean distance. Induces small but non-zero weights (Ridge). Differentiable everywhere.
Frobenius Norm	||A||_F = sqrt(sum A_ij^2). Matrix version of L2. Used in matrix factorization objectives.

1.2 Calculus & Optimization for ML

ML training = optimization. You must understand derivatives, gradients, and how to derive them for common loss functions.

Key Derivatives to Know by Heart

```
d/dx sigmoid(x) = sigmoid(x) * (1 - sigmoid(x))
```

Derivative of sigmoid. Vanishes near 0 and 1 — causes vanishing gradients.

```
d/dx tanh(x) = 1 - tanh(x)^2
```

Derivative of tanh. Also saturates but output is zero-centered.

```
d/dx ReLU(x) = 1 if x > 0 else 0
```

Sub-gradient at x=0 is typically set to 0. Does not saturate for positive values.

```
d/dw MSE = (2/N) * X^T (Xw - y)
```

Gradient of MSE loss w.r.t. weight vector w. Set to zero -> Normal Equation.

```
d/dw Cross-Entropy(sigmoid) = X^T (y_hat - y)  [per sample: (y_hat - y) * x]
```

*Elegant: gradient is simply (prediction - label) * input. No sigmoid derivative needed — they cancel out.*

Chain Rule — How Backprop Works

For a composite function f(g(x)), the derivative is: df/dx = df/dg * dg/dx. Backpropagation is just the chain rule applied repeatedly through each layer of the network.

```
  # Manual Backprop
# Manual backprop through a 2-layer network
# Forward: z1 = X @ W1 + b1; a1 = relu(z1); z2 = a1 @ W2 + b2; loss = MSE(z2, y)

# Backward (chain rule):
# dL/dz2 = 2/N * (z2 - y)                        [MSE gradient]
# dL/dW2 = a1.T @ dL_dz2                          [weight gradient]
# dL/da1 = dL_dz2 @ W2.T                          [chain rule]
# dL/dz1 = dL_da1 * (z1 > 0).astype(float)       [ReLU derivative]
# dL/dW1 = X.T @ dL_dz1
```

Convexity

A function is convex if the line segment between any two points on the curve lies above (or on) the curve. Convex functions have no local minima — only a global minimum. Linear Regression (MSE loss) is convex. Neural networks are NOT convex — many local minima exist, but in practice most local minima are close to the global minimum for overparameterized networks.

PART 2: PROBABILITY & STATISTICS (DEEP DIVE)

2. Probability & Statistics

2.1 Probability Foundations

Random Variable	A variable whose value is determined by a random process. Discrete (coin flip) or continuous (temperature). Described by a distribution.
PMF / PDF	PMF (Probability Mass Function): P(X=x) for discrete. PDF (Probability Density Function): f(x) for continuous. P(a <= X <= b) = integral f(x) dx.
CDF	F(x) = P(X <= x). Cumulative Distribution Function. Derivative of CDF = PDF.
Expectation	E[X] = sum x*P(X=x) (discrete) or integral x*f(x) dx (continuous). Linearity: E[aX + bY] = aE[X] + bE[Y].
Variance	Var(X) = E[(X - mu)^2] = E[X^2] - (E[X])^2. Var(aX) = a^2 Var(X). Var(X+Y) = Var(X)+Var(Y) only if X,Y independent.

Key Distributions (Know These Cold)

- **Bernoulli(p):** 1 coin flip. P(X=1)=p. Mean=p, Var=p(1-p). Used in logistic regression.
- **Binomial(n,p):** n independent Bernoulli trials. P(X=k) = C(n,k) p^k (1-p)^(n-k). Mean=np.
- **Poisson(λ):** Count of rare events per interval. P(X=k) = e^(-λ) λ^k / k!. Mean=Var=λ. Used for clicks, arrivals, errors.
- **Normal(μ,σ²):** Bell curve. PDF = (1/σ√2π) exp(-(x-μ)²/2σ²). 68-95-99.7 rule. Central to CLT.
- **Exponential(λ):** Time between Poisson events. PDF = λe^(-λx). Memoryless property: P(X>s+t|X>s) = P(X>t).
- **Uniform(a,b):** Equally likely everywhere. Mean=(a+b)/2. Used for random initialization.
- **Beta(α,β):** Distribution over [0,1]. Used as prior for probabilities in Bayesian inference.
- **Dirichlet(α):** Multivariate generalization of Beta. Prior for categorical/multinomial distributions.

2.2 Bayes' Theorem — Applied to ML

```
P(θ|D) = P(D|θ) * P(θ) / P(D)
```

*Posterior = Likelihood * Prior / Evidence. Read: 'probability of parameters given data = how likely is data under those params * our prior belief / normalization'*

Bayesian vs Frequentist: Frequentist: parameters are fixed, data is random. Bayesian: data is fixed, parameters are random (have distributions). Bayesian provides uncertainty estimates but is computationally harder.

Naive Bayes Classifier: Assumes features are conditionally independent given class. $P(y|x1,x2,...,xn) \propto P(y) * \prod P(xi|y)$. Surprisingly effective for text despite 'naive' assumption.

2.3 Hypothesis Testing — In Depth

Step-by-Step Hypothesis Testing

1. State H0 (null) and H1 (alternative). Example: H0: conversion rates A=B, H1: B > A
2. Choose significance level α (typically 0.05). This is your Type I error tolerance.
3. Choose the right test: t-test (compare means), chi-squared (categorical), z-test (large N), Mann-Whitney (non-parametric).
4. Compute the test statistic and p-value.
5. If p-value < α: reject H0. If p-value ≥ α: fail to reject H0 (NOT the same as accepting H0).
6. Report effect size (not just p-value). Cohen's d for t-test: (μ1 - μ2) / pooled_std.

Common Tests

t-test (independent)	Compare means of two groups. Assumes normality and equal variance (Welch's t-test doesn't require equal variance). $t = (\bar{x}1 - \bar{x}2) / sqrt(s1^2/n1 + s2^2/n2)$.
Paired t-test	Compare same subjects under two conditions (e.g., before/after). More powerful because within-subject variation is removed.
Chi-squared test	Test independence of categorical variables (e.g., click rate by button color). $\chi^2 = sum (observed - expected)^2 / expected$.
KS Test	Kolmogorov-Smirnov: tests if two samples come from the same distribution. Non-parametric. Used for detecting data drift in production.
Mann-Whitney U	Non-parametric alternative to t-test. Tests if one distribution tends to have larger values. Use when normality assumption fails.

Power Analysis — Sample Size Calculation

Statistical Power (1-β): Probability of correctly detecting a real effect. Standard: 80% or 90% power. Low power = high risk of missing real improvements.

Key factors: Effect size (bigger effect = fewer samples needed), α (lower α = more samples), power (higher power = more samples).

```python
# A/B Test: Power Analysis & Significance Test
from scipy import stats
import numpy as np

# Two-proportion z-test: A/B test on conversion rate
# H0: p_control = p_treatment
control_rate = 0.10   # 10% baseline conversion
mde = 0.02            # minimum detectable effect: +2% lift
alpha = 0.05
power = 0.80

# Using statsmodels for sample size
from statsmodels.stats.proportion import proportion_effectsize, zt_ind_solve_power
effect = proportion_effectsize(control_rate, control_rate + mde)
n = zt_ind_solve_power(effect_size=effect, alpha=alpha, power=power, alternative='two-sided')
print(f'Required n per group: {int(np.ceil(n))}')  # e.g., ~3800 per group

# Run the test after collecting data
successes_A, n_A = 980, 10000
successes_B, n_B = 1050, 10000
stat, p_value = stats.proportions_ztest([successes_A, successes_B], [n_A, n_B])
print(f'p-value: {p_value:.4f}')  # < 0.05 -> significant
```

Simpson's Paradox — MUST KNOW for Interviews

A trend appears in several groups but reverses when the groups are combined. Example: Treatment A has a higher recovery rate in both mild AND severe cases, but Treatment B has a higher overall recovery rate because it was applied to more mild cases (who recover easily regardless).

Implication for ML: Aggregate metrics can be misleading. Always segment analysis by relevant subgroups. A model that improves overall accuracy may degrade performance on minority subgroups.

2.4 Information Theory

Entropy: H(X) = -sum P(x) log P(x)

Measures uncertainty/disorder. Max entropy = uniform distribution. Zero entropy = deterministic. Used in decision tree splitting (information gain).

KL Divergence: KL(P||Q) = sum P(x) log(P(x)/Q(x))

Measures how different distribution Q is from reference P. Non-symmetric: KL(P||Q) ≠ KL(Q||P). Not a true distance. Used in VAEs, RL.

Cross-Entropy: H(P,Q) = -sum P(x) log Q(x) = H(P) + KL(P||Q)

When P is the true distribution (labels) and Q is the model's predicted distribution. Minimizing cross-entropy = minimizing KL divergence from truth. This is why we use cross-entropy loss.

Mutual Information: I(X;Y) = H(X) - H(X|Y)

Measures how much knowing Y reduces uncertainty about X. Used for feature selection and understanding dependencies.

2.5 Causal Inference — Critical for Amazon/Microsoft

Correlation ≠ causation. Causal inference asks: 'What WOULD happen if we intervened?' Applied scientists need this for A/B testing, policy decisions, and avoiding spurious correlations.

Key Concepts

Confounders	Variables that affect both treatment and outcome, creating spurious correlation. Example: exercise and health — healthier people may both exercise more AND have fewer diseases, making it hard to isolate exercise's effect.
Counterfactual	What would have happened to a unit had it received a different treatment? Fundamental problem: we can only observe one potential outcome per unit. ATE = E[Y(1) - Y(0)] = Average Treatment Effect.
Propensity Score	P(T=1|X): probability of treatment given covariates. Match or reweight treated/control units with similar propensity scores to reduce confounding bias. Assumes no hidden confounders.
Diff-in-Differences	Compare changes over time between treatment and control groups. Controls for time-invariant confounders. DiD = (Y_treat_after - Y_treat_before) - (Y_control_after - Y_control_before).
Instrumental Variable	A variable that affects treatment but does not directly affect outcome. Breaks confounding. Hard to find good instruments in practice. Example: distance to hospital as IV for effect of hospitalization.
do-calculus	Pearl's framework for computing causal effects from observational data using a causal graph (DAG). P(Y|do(X)) is not the same as P(Y|X).

PART 3: MACHINE LEARNING ALGORITHMS — DEEP DIVE

3. Classical ML Algorithms — Mathematical & Practical

3.1 Linear Regression — Full Derivation

Model: $\hat{y} = Xw + b$. Loss: MSE = $(1/N)||y - Xw||^2$. Closed-form: $w^* = (X^T X)^{-1} X^T y$ (Normal Equation). Problems: $O(d^3)$ inversion, fails if $X^T X$ is singular (multicollinear features). Fix: add λI (Ridge) → $w^* = (X^T X + \lambda I)^{-1} X^T y$.

```
Gradient of MSE: ∇_w L = (2/N) X^T (Xw - y)
```

Derived by expanding $||y - Xw||^2 = y^T y - 2w^T X^T y + w^T X^T Xw$, then differentiating.

Assumptions of Linear Regression (BLUE):

- Linearity: relationship between X and y is linear
- Independence: residuals are independent of each other (no autocorrelation)
- Homoscedasticity: residuals have constant variance (no heteroscedasticity)
- Normality of residuals: needed for valid confidence intervals and hypothesis tests
- No perfect multicollinearity: no feature is a perfect linear combination of others

3.2 Logistic Regression — Full Derivation

```
P(y=1|x) = σ(w^T x + b) = 1 / (1 + e^{-z})
```

The log-odds (logit) is linear in x. Decision boundary: $w^T x + b = 0$.

```
Loss (Binary Cross-Entropy): L = -(1/N) Σ [y log(ŷ) + (1-y) log(1-ŷ)]
```

This is the MLE objective under Bernoulli distribution. No closed-form solution — solved iteratively with gradient descent.

```
Gradient: ∂L/∂w = (1/N) X^T (ŷ - y)
```

*Elegant result: gradient = prediction error * input features. Same form as linear regression gradient!*

Multinomial Logistic Regression: Uses softmax instead of sigmoid. For K classes: P(y=k|x) = exp(w_k^T x) / Σ_j exp(w_j^T x). Loss = multi-class cross-entropy.

3.3 Decision Trees — From Scratch

A greedy algorithm that recursively partitions the feature space. At each node, it tries all features and all split points, choosing the split that maximizes purity gain.

Gini Impurity: $G = 1 - \Sigma\, p_k^2$

pk = fraction of class k at the node. G=0: pure node. G=0.5: maximally mixed (2 classes). Split reduces weighted Gini across children.

Information Gain: IG = H(parent) - [N_L/N * H(left) + N_R/N * H(right)]

How much does the split reduce entropy? Higher = better split. CART uses Gini; ID3/C4.5 use entropy.

```
# Gini Impurity and Best Split Implementation
import numpy as np

def gini(y):
    classes, counts = np.unique(y, return_counts=True)
    p = counts / len(y)
    return 1 - np.sum(p**2)

def best_split(X, y):
    best_gain, best_feat, best_thresh = -1, None, None
    parent_gini = gini(y)
    for feat in range(X.shape[1]):
        thresholds = np.unique(X[:, feat])
        for thresh in thresholds:
            left_mask = X[:, feat] <= thresh
            right_mask = ~left_mask
            if left_mask.sum() == 0 or right_mask.sum() == 0: continue
            n, n_l, n_r = len(y), left_mask.sum(), right_mask.sum()
            weighted_gini = (n_l/n)*gini(y[left_mask]) + (n_r/n)*gini(y[right_mask])
            gain = parent_gini - weighted_gini
            if gain > best_gain:
                best_gain, best_feat, best_thresh = gain, feat, thresh
    return best_feat, best_thresh, best_gain
```

Pruning

- **Pre-pruning (early stopping):** Set max_depth, min_samples_split, min_samples_leaf. Stops the tree from growing too deep.
- **Post-pruning (cost-complexity):** Grow full tree, then remove subtrees that don't improve performance on validation set. Sklearn: ccp_alpha parameter.

3.4 Ensemble Methods — Detailed

Bagging (Bootstrap Aggregating)

Train B independent base learners on B bootstrap samples (random samples with replacement). Each sample uses ~63.2% of unique training points (the rest are 'out-of-bag' or OOB). Prediction = majority vote or average.

Why does it work? Each learner has high variance (overfits). Averaging uncorrelated learners reduces variance by factor ~B. Key: the learners must be diverse (hence randomness in sampling).

Random Forest — Specific Enhancements

- At each split, consider only sqrt(d) features (classification) or d/3 features (regression). This **decorrelates** the trees — makes averaging more effective.
- **OOB Error:** Each tree's OOB samples act as a built-in validation set. A free performance estimate without CV. Usually close to 5-fold CV.
- **Feature Importance:** Mean Decrease in Impurity (MDI) across all splits using that feature. WARNING: biased toward high-cardinality features. Use permutation importance instead for fair comparison.

```
# Random Forest with OOB and Permutation Importance
from sklearn.ensemble import RandomForestClassifier
from sklearn.inspection import permutation_importance

rf = RandomForestClassifier(n_estimators=300, max_features='sqrt',
                            oob_score=True, n_jobs=-1, random_state=42)
rf.fit(X_train, y_train)
print(f'OOB score: {rf.oob_score_:.4f}')       # free validation estimate

# Permutation importance (more reliable than MDI)
result = permutation_importance(rf, X_val, y_val, n_repeats=10, random_state=42)
sorted_idx = result.importances_mean.argsort()[::-1]
for i in sorted_idx[:10]:
    print(f'{feature_names[i]}: {result.importances_mean[i]:.4f}')
```

Gradient Boosting — Mathematical Derivation

Key insight: fit each new tree to the **negative gradient** of the loss function. For MSE loss, negative gradient = residuals. For other losses, it's a pseudo-residual.

$$F_m(x) = F_{m-1}(x) + \eta * h_m(x)$$

η = learning rate (shrinkage). h_m = new tree fitted to residuals $r_i = -[\partial L/\partial F(x_i)]$ at $F = F_{m-1}$.

XGBoost enhancements over vanilla GBM:

- Uses **second-order Taylor expansion** of loss → better tree objective
- **L1 + L2 regularization** on leaf weights (alpha, lambda params)
- **Column subsampling** per tree and per split (like Random Forest)
- **Sparsity-aware split finding** → handles missing values natively
- **Approximate greedy algorithm** for finding splits efficiently on large data
- **Parallel and distributed** computation using sorted blocks

```
# XGBoost and LightGBM
import xgboost as xgb
import lightgbm as lgb

# XGBoost with early stopping
dtrain = xgb.DMatrix(X_train, label=y_train)
dval   = xgb.DMatrix(X_val, label=y_val)
params = {
```

```
    'max_depth': 6, 'eta': 0.05, 'subsample': 0.8,
    'colsample_bytree': 0.8, 'min_child_weight': 3,
    'lambda': 1.0, 'alpha': 0.0,  # L2, L1 regularization
    'objective': 'binary:logistic', 'eval_metric': 'auc',
}
model = xgb.train(params, dtrain, num_boost_round=1000,
                  evals=[(dval, 'val')], early_stopping_rounds=50)

# LightGBM: leaf-wise growth (faster, better for large datasets)
lgb_model = lgb.LGBMClassifier(n_estimators=500, learning_rate=0.05,
                               num_leaves=63, min_child_samples=20,
                               subsample=0.8, colsample_bytree=0.8)
```

Stacking (Stacked Generalization)

Train diverse base models (level-0), then train a meta-learner (level-1) on their out-of-fold predictions. Meta-learner learns how to best combine base model outputs. Often outperforms any individual model.

```
  # Stacking Ensemble
from sklearn.ensemble import StackingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.svm import SVC

estimators = [
    ('rf', RandomForestClassifier(n_estimators=100)),
    ('gb', GradientBoostingClassifier(n_estimators=100)),
    ('svm', SVC(probability=True))
]
stacking = StackingClassifier(
    estimators=estimators,
    final_estimator=LogisticRegression(),  # meta-learner
    cv=5,  # out-of-fold predictions for meta-learner training
    passthrough=False  # set True to also pass original features to meta-learner
)
```

3.5 Support Vector Machines — Deep Dive

Primal SVM: min (1/2)||w||² s.t. y_i(w^T x_i + b) >= 1 ∀i

Find the hyperplane with maximum margin (= 2/||w||). Constraints ensure all points are correctly classified with margin.

Soft-margin SVM: min (1/2)||w||² + C Σ ξ_i s.t. y_i(w^T x_i + b) >= 1 - ξ_i

Slack variables ξ_i allow misclassification. C controls trade-off: large C = less margin, fewer errors; small C = larger margin, more errors tolerated.

Kernel Functions:

- Linear: K(x,z) = x^T z. No transformation. Fast. Best for linearly separable data.
- RBF (Gaussian): K(x,z) = exp(-γ||x-z||²). Maps to infinite-dimensional space. Most powerful. Two hyperparameters: C and γ.
- Polynomial: K(x,z) = (x^T z + r)^d. Good for image classification.

Dual Formulation: The dual problem depends only on dot products $K(x_i, x_j)$ — enabling the kernel trick. At prediction time: $f(x) = \text{sign}(\Sigma\ \alpha_i\ y_i\ K(x_i, x) + b)$. Only support vectors ($\alpha_i > 0$) matter.

3.6 Unsupervised Learning — Complete

K-Means — Algorithm and Limitations

7. Randomly initialize K centroids (K-Means++ for better init: place centroids far from each other)
8. Assign each point to the nearest centroid (E-step)
9. Update centroids to the mean of their assigned points (M-step)
10. Repeat until convergence (centroid positions stabilize)

Choosing K: Elbow method (plot inertia vs K, pick 'elbow'), Silhouette score (measures how similar point is to own cluster vs nearest other cluster, range [-1,1]), Gap statistic (compare inertia to random data).

Limitations: Assumes spherical clusters, sensitive to scale (always normalize!), sensitive to initialization, doesn't handle different cluster sizes or densities well.

Hierarchical Clustering

Agglomerative: start with each point as its own cluster, merge closest pairs. Creates a dendrogram. Cut at desired level to get K clusters. No need to specify K upfront. Linkage methods: single (min dist), complete (max dist), average, ward (minimize variance increase).

Gaussian Mixture Models (GMM)

Soft clustering: each point has a probability of belonging to each cluster. Model: $P(x) = \Sigma_k\ \pi_k\ N(x; \mu_k, \Sigma_k)$. Parameters estimated via EM algorithm. More flexible than K-Means (allows elliptical clusters). Generative model — can sample new points.

```
  # GMM with BIC for model selection
from sklearn.mixture import GaussianMixture
from sklearn.metrics import silhouette_score

# Select number of components using BIC (lower = better)
bic_scores = []
for k in range(2, 11):
    gmm = GaussianMixture(n_components=k, covariance_type='full', random_state=42)
    gmm.fit(X)
    bic_scores.append(gmm.bic(X))
best_k = bic_scores.index(min(bic_scores)) + 2

gmm = GaussianMixture(n_components=best_k, random_state=42)
labels = gmm.fit_predict(X)
probabilities = gmm.predict_proba(X)  # soft assignments
```

3.7 Anomaly Detection

Isolation Forest	Randomly partitions data by selecting a feature and split value. Anomalies require fewer splits to isolate (shorter path length). Efficient O(n log n). Works well in high dimensions.
One-Class SVM	Learns a boundary around normal data. Points outside the boundary are anomalies. Suitable for high-dimensional data. Slower than Isolation Forest.

Autoencoder for Anomaly	Train autoencoder on normal data. Anomalies will have high reconstruction error because the model hasn't learned to compress them. Very effective for images, sensor data.

LOF (Local Outlier Factor)	Compare density of a point to its k-nearest neighbors. Points in low-density regions compared to their neighbors are outliers. LOF score >> 1 = anomaly. Good for density-based anomalies.

PART 4: DEEP LEARNING — COMPREHENSIVE

4. Deep Learning — Architecture to Training to Production

4.1 Weight Initialization

Poor initialization = symmetry (all neurons learn the same thing), vanishing/exploding gradients. This is critical but often overlooked.

Xavier (Glorot) Init	Var(W) = 2/(fan_in + fan_out). Designed for sigmoid/tanh. Keeps variance of activations and gradients stable across layers.

He (Kaiming) Init	Var(W) = 2/fan_in. Designed for ReLU. Accounts for the fact that ReLU zeros out half the neurons. Use by default for networks with ReLU/LeakyReLU.

Zero Init Warning	NEVER initialize all weights to zero — all neurons will compute the same output and receive the same gradient. Symmetry breaking requires randomness.

```
# Weight Initialization Best Practice
import torch.nn as nn

def init_weights(m):
    if isinstance(m, nn.Linear):
        nn.init.kaiming_normal_(m.weight, mode='fan_in', nonlinearity='relu')
        nn.init.zeros_(m.bias)
    elif isinstance(m, nn.Conv2d):
        nn.init.kaiming_normal_(m.weight, mode='fan_out', nonlinearity='relu')
    elif isinstance(m, nn.BatchNorm2d):
        nn.init.ones_(m.weight)   # gamma = 1
        nn.init.zeros_(m.bias)    # beta = 0

model.apply(init_weights)
```

4.2 Optimizers — Complete Math

SGD with Momentum

$$v = \beta * v - \eta * \nabla L; \quad \theta = \theta + v$$

β (momentum) typically 0.9. Accumulates gradient history. Reduces oscillations. Can escape shallow local minima.

Adam — Step by Step

```
m_t = β1*m_{t-1} + (1-β1)*g_t   [1st moment — mean]
```

β1=0.9 typically. Exponential moving average of gradients.

```
v_t = β2*v_{t-1} + (1-β2)*g_t²  [2nd moment — uncentered variance]
```

β2=0.999 typically. Exponential moving average of squared gradients.

```
m^_t = m_t/(1-β1^t);  v^_t = v_t/(1-β2^t)  [bias correction]
```

Early steps have small m and v — bias correction prevents underestimation.

```
θ_t = θ_{t-1} - η * m^_t / (sqrt(v^_t) + ε)
```

Per-parameter adaptive learning rate. Features with sparse gradients get larger updates. η=1e-3, ε=1e-8.

AdamW: Decoupled weight decay. Instead of adding $\lambda||w||^2$ to loss (which interacts with Adam's adaptive rates), directly penalize weights: $\theta = \theta - \eta*(\hat{m}/\text{sqrt}(\hat{v}) + \lambda\theta)$. Preferred for transformers.

When to use what: Adam/AdamW for transformers and NLP. SGD+momentum for CNNs (often achieves better generalization with proper LR scheduling). Adagrad/Adadelta for sparse features (NLP with simpler models).

4.3 Learning Rate Scheduling

Step Decay	Reduce LR by factor every N epochs. Simple. lr = lr_0 * decay^floor(epoch/step_size).
Cosine Annealing	LR follows cosine curve from max to min. Smooth. Often outperforms step decay. With warm restarts (SGDR): periodic resets to explore different minima.
Linear Warmup	Start with very small LR, increase linearly for first N steps, then decay. Standard for transformers. Prevents large gradient steps early when model is random.
One Cycle Policy	LR increases then decreases within one cycle. Momentum does the opposite. Proposed by fast.ai. Can train very fast.

```
# LR Scheduling
from torch.optim.lr_scheduler import CosineAnnealingLR, OneCycleLR, LinearLR, SequentialLR

optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=0.01)

# Linear warmup + cosine decay (standard for transformers)
warmup = LinearLR(optimizer, start_factor=0.01, end_factor=1.0, total_iters=1000)
```

```
cosine = CosineAnnealingLR(optimizer, T_max=9000)
scheduler = SequentialLR(optimizer, schedulers=[warmup, cosine], milestones=[1000])

# Or use transformers library scheduler
from transformers import get_cosine_schedule_with_warmup
scheduler = get_cosine_schedule_with_warmup(
    optimizer, num_warmup_steps=500, num_training_steps=10000
)
```

4.4 CNN Architecture Deep Dive

Convolution Mathematics

Output size: O = floor((I + 2P - K) / S) + 1

I=input size, P=padding, K=kernel size, S=stride. Example: 32x32 input, 3x3 kernel, stride=1, padding=1 -> output=32x32.

Parameters in Conv layer: K*K*C_in*C_out + C_out (bias)

*Example: 3x3 conv, 64->128 channels = 3*3*64*128 + 128 = 73,856 params.*

ResNet — Residual Learning

Key insight: learning residual F(x) = H(x) - x is easier than learning H(x) directly. Identity shortcut provides gradient highway. Enables training 100-1000 layer networks.

```
# Residual Block
class ResidualBlock(nn.Module):
    def __init__(self, channels):
        super().__init__()
        self.conv1 = nn.Conv2d(channels, channels, 3, padding=1, bias=False)
        self.bn1   = nn.BatchNorm2d(channels)
        self.conv2 = nn.Conv2d(channels, channels, 3, padding=1, bias=False)
        self.bn2   = nn.BatchNorm2d(channels)
    def forward(self, x):
        residual = x
        out = F.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        return F.relu(out + residual)  # skip connection
```

Depthwise Separable Convolutions (MobileNet)

Split standard convolution into: (1) Depthwise conv: apply one filter per input channel (K*K*C_in params). (2) Pointwise conv: 1x1 conv to combine channels (C_in*C_out params). Result: ~8-9x fewer parameters than standard conv with similar accuracy. Foundation of efficient mobile networks.

4.5 Transformer Architecture — Complete

Step-by-Step Token Journey

11. Input tokens → Embedding lookup (vocab_size → d_model)
12. Add positional encoding (same shape as embeddings, added element-wise)
13. Pass through N encoder/decoder blocks in sequence
14. Each block: LayerNorm → Multi-Head Attention → Residual → LayerNorm → FFN → Residual

15. Final output → projection to vocab size → softmax → next token probability

Attention — Full Math

```
Q = XW_Q,  K = XW_K,  V = XW_V  (linear projections)
```

W_Q, W_K, W_V ∈ R^{d_model × d_k}. Different heads use different projection matrices.

```
Attention(Q,K,V) = softmax(QK^T / sqrt(d_k)) * V
```

d_k = d_model / num_heads. Division by sqrt(d_k) prevents softmax from saturating (too-peaked distributions = vanishing gradients).

```
MultiHead(Q,K,V) = Concat(head_1,...,head_h) W_O
```

Each head attends to different parts of the sequence. W_O ∈ R^{h*d_v × d_model} projects back to d_model.

Complexity: Self-attention is $O(N^2d)$ in time and $O(N^2)$ in space for sequence length N. This is why LLMs have context length limits. For N=4096, N^2 = 16M attention weights per head!

Flash Attention — Efficient Attention

Standard attention materializes the full N×N attention matrix in GPU HBM (slow). Flash Attention uses tiling to compute attention in SRAM blocks without materializing the full matrix, achieving O(N) memory instead of $O(N^2)$. Enables 2-4x speedup and much longer context windows. Mathematically identical to standard attention.

Feed-Forward Network

```
FFN(x) = max(0, xW_1 + b_1) W_2 + b_2
```

*Two linear layers with ReLU (or GELU). Inner dimension = 4*d_model typically. Contains ~2/3 of transformer parameters! Acts as key-value memory.*

Positional Encodings

Sinusoidal (Original)	PE(pos, 2i) = sin(pos/10000^{2i/d_model}). Fixed, not learned. Allows generalization to unseen lengths. Each position gets a unique pattern.
Learned Absolute PE	Train embeddings for each position. Simple but doesn't generalize to lengths not seen in training.
RoPE (Rotary PE)	Rotate query/key vectors by angle proportional to position. Relative positions preserved in dot products. Used in LLaMA, GPT-NeoX. Best current approach.
ALiBi	Adds a linear bias to attention scores based on distance. Very simple. Strong length generalization. Used in BLOOM.

4.6 Large Language Models — Fine-tuning

Full Fine-tuning vs Parameter-Efficient Fine-tuning (PEFT)

Full fine-tuning updates ALL parameters. Expensive for large models (GPT-3: 175B params). PEFT methods add or modify a small fraction of parameters:

LoRA (Low-Rank Adaptation)	Freeze pre-trained weights W. Add low-rank update: W' = W + BA where B∈R^{d×r}, A∈R^{r×k}, r << min(d,k). Only A and B are trained. Typical r=8-64 vs d=4096. Reduces trainable params by 10000x. Quality close to full fine-tuning.
QLoRA	LoRA + Quantize base model to 4-bit (NF4 format). Enables fine-tuning 65B models on a single A100 GPU. Used for most LLM fine-tuning today.
Prefix Tuning	Prepend learnable 'virtual tokens' to input. Only train these prefix parameters (~0.1% of params). Good for generation tasks.
Prompt Tuning	Even simpler than prefix tuning: only train the embedding of a soft prompt. Very parameter-efficient but requires large base models to work well.

```
  # LoRA Fine-tuning with PEFT
from peft import LoraConfig, get_peft_model, TaskType
from transformers import AutoModelForCausalLM

base_model = AutoModelForCausalLM.from_pretrained('meta-llama/Llama-2-7b-hf')

lora_config = LoraConfig(
    r=16,                          # rank
    lora_alpha=32,                 # scaling: alpha/r = effective LR multiplier
    target_modules=['q_proj', 'v_proj'],  # apply LoRA to Q and V projections
    lora_dropout=0.1,
    bias='none',
    task_type=TaskType.CAUSAL_LM
)
model = get_peft_model(base_model, lora_config)
model.print_trainable_parameters()
# trainable params: 4,194,304 || all params: 6,742,609,920 || trainable%: 0.0622%
```

RLHF — Reinforcement Learning from Human Feedback

Used to align LLMs with human preferences (ChatGPT, Claude). Three stages:

16. **Supervised Fine-Tuning (SFT):** Fine-tune base LLM on demonstrations of desired behavior (instruction-response pairs).
17. **Reward Model Training:** Collect human preference comparisons (response A vs B). Train a classifier to predict which response is preferred. Output is a scalar reward score.
18. **PPO (Proximal Policy Optimization):** Use the reward model to score LLM responses. Optimize the LLM policy with PPO to maximize reward, with a KL penalty to prevent it drifting too far from the SFT model. KL penalty prevents reward hacking.

DPO (Direct Preference Optimization): Newer alternative to RLHF-PPO. Skips explicit reward model — directly optimizes LLM on preference data. Simpler to implement, often comparable quality.

4.7 Debugging Deep Learning Models

This is heavily tested in interviews: 'Your model is not converging. What do you do?'

Systematic Debugging Process

19. **Overfit a single batch first.** If your model can't perfectly overfit 10 samples, something is fundamentally wrong (bugs in architecture, loss function, data loading).
20. **Check data:** Visualize samples. Verify labels are correct. Check class distribution. Verify normalization.
21. **Check gradients:** Log gradient norms per layer. NaN = exploding (add gradient clipping). Near-zero = vanishing (check activations, skip connections, BatchNorm).
22. **Check loss:** Plot training loss curve. Flat = LR too small, gradient issue, or wrong loss. Oscillating = LR too large. Diverging = LR way too large.
23. **Monitor activations:** Dead ReLU neurons (always zero output), exploding pre-activations. Use hooks in PyTorch.
24. **Simplify then scale:** Start with smallest reasonable model. Confirm it works. Scale up.

```
  # Gradient Monitoring & Clipping
# Gradient monitoring in PyTorch
for name, param in model.named_parameters():
    if param.grad is not None:
        grad_norm = param.grad.norm().item()
        if grad_norm != grad_norm:  # NaN check
            print(f'NaN gradient in {name}!')
        elif grad_norm > 100:
            print(f'Exploding gradient in {name}: {grad_norm:.2f}')
        elif grad_norm < 1e-7:
            print(f'Vanishing gradient in {name}: {grad_norm:.2e}')

# Gradient clipping to prevent explosion
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
```

PART 5: NLP — FROM TOKENIZATION TO LLMS

5. Natural Language Processing — Comprehensive

5.1 Tokenization — How Text Becomes Tokens

Word Tokenization	Split by spaces/punctuation. Problem: 'running' and 'run' are different tokens. Huge vocabulary. Fails on unseen words.
Byte-Pair Encoding (BPE)	Start with character vocabulary. Iteratively merge most frequent adjacent pairs. GPT-2/3/4 use BPE. Good balance: handles morphology, reasonable vocab size (~50K).
WordPiece	Similar to BPE but maximizes likelihood of the training data. Used by BERT. Unknown words split into '##' subwords: 'unbelievable' → ['un', '##believ', '##able'].
SentencePiece	Language-agnostic. Works directly on raw bytes. No need for pre-tokenization. Used by T5, LLaMA.

Why tokenization matters: A model's context window is in TOKENS, not words. 1 English word ≈ 1.3 tokens. Code/numbers can be 2-5 tokens per word. Arithmetic is hard because numbers split into digits (3-5 tokens each).

5.2 Language Model Fundamentals

```
P(w_1, w_2, ..., w_n) = Π P(w_t | w_1, ..., w_{t-1})
```

A language model assigns probabilities to sequences. Autoregressive (causal) models generate by sampling from this distribution left-to-right.

Perplexity	PP(W) = 2^{H(W)} = 2^{-(1/N) Σ log2 P(w_t|context)}. Lower perplexity = better LM. Perplexity of 10 means the model is as confused as if choosing uniformly among 10 options at each step.

Masked LM (BERT): Randomly mask 15% of tokens. Predict masked tokens using BOTH left and right context. Non-causal. Better for understanding tasks.

Causal LM (GPT): Predict next token given all previous tokens. Causal masking prevents seeing future tokens. Used for generation. Scale to GPT-4, LLaMA, Claude.

5.3 BERT Fine-tuning Patterns

Task Heads on Top of BERT

- **Text Classification:** Use [CLS] token embedding → Linear → softmax. [CLS] aggregates sequence-level information.
- **Token Classification (NER):** Use all token embeddings → Linear per token → softmax. Predict tag for each token.
- **Question Answering (Extractive):** Two linear classifiers over all tokens: predict start position and end position of answer span.
- **Sentence Pair (NLI, STS):** Concatenate sentences with [SEP], use [CLS] for classification or similarity score.

```
  # BERT Fine-tuning for Classification
from transformers import BertTokenizer, BertForSequenceClassification
import torch

tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
model = BertForSequenceClassification.from_pretrained(
    'bert-base-uncased', num_labels=2
)

# Tokenize input
inputs = tokenizer(
    ['This movie is great!', 'I hated it.'],
    padding=True, truncation=True, max_length=128,
    return_tensors='pt'
)
outputs = model(**inputs)              # logits: (batch, 2)
probs = torch.softmax(outputs.logits, dim=-1)

# Fine-tuning with AdamW + warmup
from transformers import AdamW, get_linear_schedule_with_warmup
optimizer = AdamW(model.parameters(), lr=2e-5, weight_decay=0.01)
# Common BERT fine-tuning: 2e-5 to 5e-5 LR, 2-4 epochs, batch=32
```

5.4 Text Embeddings & Semantic Search

Sentence Transformers (SBERT): Fine-tunes BERT with contrastive loss (triplet/cosine similarity) to produce sentence-level embeddings where semantic similarity corresponds to cosine distance. Enables fast semantic search using ANN.

```
  # Semantic Search with SBERT + FAISS
from sentence_transformers import SentenceTransformer, util
import faiss, numpy as np

model = SentenceTransformer('all-MiniLM-L6-v2')

# Encode corpus
corpus = ['Machine learning is great', 'Deep learning uses neural networks', ...]
embeddings = model.encode(corpus, batch_size=64, normalize_embeddings=True)

# Build FAISS index for fast ANN search
d = embeddings.shape[1]
index = faiss.IndexFlatIP(d)  # Inner product (= cosine for normalized vecs)
index.add(embeddings.astype('float32'))
```

```
# Query
query = model.encode(['What is deep learning?'], normalize_embeddings=True)
distances, indices = index.search(query.astype('float32'), k=5)
```

5.5 RAG — Retrieval Augmented Generation

LLMs have fixed knowledge (training cutoff). RAG grounds responses in a dynamic knowledge base. Architecture: (1) Encode documents into embeddings and store in vector DB. (2) At query time, retrieve top-K relevant docs. (3) Concatenate docs + question as context for LLM.

- **Chunking strategy:** Split docs into overlapping chunks (512-1024 tokens with 10-20% overlap). Smaller = more precise retrieval. Larger = more context per chunk.
- **Hybrid retrieval:** Combine dense retrieval (embeddings, semantic) with sparse retrieval (BM25, keyword). BM25 good for exact term matching; dense good for semantic understanding. RRF (Reciprocal Rank Fusion) to merge results.
- **Re-ranking:** Use a cross-encoder to re-rank top-K retrieved docs. More accurate than bi-encoder but too slow for full corpus.
- **Hallucination risk:** Always add source attribution. Check if LLM answer is grounded in retrieved context (faithfulness score).

5.6 Evaluation of NLP Models

BLEU	Precision of n-grams in generated text compared to reference(s). Range [0,1]. Used for machine translation. Limitations: doesn't measure recall, poor correlation with human judgment for open-ended generation.
ROUGE	Recall-oriented. ROUGE-N: n-gram recall. ROUGE-L: longest common subsequence. Standard for summarization evaluation.
BERTScore	Compute cosine similarity between generated and reference embeddings using BERT. Better correlation with human judgment than BLEU/ROUGE. Captures semantic equivalence.
Perplexity	Intrinsic metric for LMs. Low perplexity = model assigns high probability to test data. Doesn't always correlate with downstream task performance.

PART 6: ML SYSTEM DESIGN — INTERVIEW FRAMEWORK

6. ML System Design — Production-Ready Thinking

6.1 The Complete System Design Framework

Use this framework for every system design question. Interviewers want to see structured thinking, not just technical knowledge.

25. **Clarify Requirements (5 min):** Scale (MAU, QPS), latency SLA, freshness needs, budget, regulatory constraints, success metrics.
26. **Problem Formulation (5 min):** ML task type, label definition, objective function, constraints (fairness, latency).
27. **Data (10 min):** Sources, volume, labels, collection strategy, potential biases, train/val/test splits.
28. **Feature Engineering (10 min):** Raw features → engineered features, embeddings. Feature store? Real-time vs batch?
29. **Model (10 min):** Start simple (logistic regression baseline). Justify complexity additions. Architecture trade-offs.
30. **Evaluation (5 min):** Offline metrics (AUC, F1, NDCG). Online metrics (CTR, revenue, engagement). Guardrail metrics.
31. **Serving & Infrastructure (10 min):** Batch vs real-time. Latency requirements. Model compression. A/B testing.
32. **Monitoring & Iteration (5 min):** Data drift, concept drift. Retraining triggers. Feedback loops.

6.2 Design: Recommendation System (Amazon/Netflix Scale)

Two-Stage Architecture (Industry Standard)

Stage 1 — Candidate Retrieval: Narrow millions of items to hundreds of candidates efficiently. Speed over precision.

- Collaborative Filtering: Matrix Factorization (ALS, SVD). Fast retrieval via ANN on user/item embeddings.
- Two-Tower Model: User encoder + Item encoder. Train on positive (click/purchase) vs negative (random) samples. At serving: pre-compute item embeddings, retrieve via FAISS.
- Graph-based: GraphSAGE on user-item interaction graph. Captures higher-order interactions.

Stage 2 — Ranking: Score and rank the ~500 candidates. Quality over speed.

- Features: user history, item features, contextual (time, device, session), cross features (user x item).
- Model: Wide & Deep (wide = memorization via logistic regression; deep = generalization via DNN). DIN (attention over user history). DLRM.
- Loss: binary cross-entropy on clicks, or pairwise ranking loss (BPR).

Key Design Decisions

Explore vs Exploit	Pure exploitation = filter bubble. Add exploration: epsilon-greedy (show random items with prob ε), Thompson Sampling (Bayesian), UCB (Upper Confidence Bound). Bandit algorithms balance exploration and exploitation.
Diversity	Avoid showing 10 similar items. Use maximal marginal relevance (MMR) or diversification post-processing. Diversity improves long-term engagement.
Position Bias	Users click higher results regardless of relevance. Debias with: inverse propensity scoring, position-aware model (add position as feature), pairwise debiasing.

```
  # Two-Tower Model
# Two-Tower model skeleton
class TwoTowerModel(nn.Module):
    def __init__(self, user_dim, item_dim, embed_dim=128):
        super().__init__()
        self.user_tower = nn.Sequential(
            nn.Linear(user_dim, 256), nn.ReLU(),
            nn.Linear(256, embed_dim)
        )
        self.item_tower = nn.Sequential(
            nn.Linear(item_dim, 256), nn.ReLU(),
            nn.Linear(256, embed_dim)
        )
    def forward(self, user_feats, item_feats):
        u = F.normalize(self.user_tower(user_feats), dim=-1)
        v = F.normalize(self.item_tower(item_feats), dim=-1)
        return (u * v).sum(-1)  # cosine similarity

# In-batch negative sampling (efficient training trick):
# For batch of (user_i, item_i) positive pairs,
# treat (user_i, item_j for j!=i) as negatives -- O(B^2) negatives per batch
```

6.3 Design: Search Ranking System

Learning to Rank (LTR)

Pointwise	Predict relevance score for each query-doc pair independently. Simple regression/classification. Ignores relative ordering. Doesn't directly optimize ranking metrics.
Pairwise (RankNet, LambdaRank)	Compare pairs of documents. Minimize inversions in ordering. LambdaRank: scale gradients by change in NDCG. LambdaMART = LambdaRank + gradient boosting. Industry workhorse.
Listwise (SoftRank, ListNet)	Optimize directly over the full ranked list. Directly maximizes NDCG/MAP. More complex but better theoretical foundation.

NDCG — Key Ranking Metric

```
NDCG@K = DCG@K / IDCG@K
```

DCG@K = Σ_{i=1}^{K} (2^rel_i - 1) / log2(i+1). IDCG = DCG of ideal ordering. Range [0,1]. Penalizes relevant docs ranked lower.

```
# NDCG Implementation
def dcg_at_k(rels, k):
    rels = np.array(rels[:k])
    if len(rels) == 0: return 0.0
    gains = (2**rels - 1) / np.log2(np.arange(2, len(rels)+2))
    return gains.sum()

def ndcg_at_k(rels, k):
    dcg = dcg_at_k(rels, k)
    ideal = dcg_at_k(sorted(rels, reverse=True), k)
    return dcg / ideal if ideal > 0 else 0.0
```

6.4 Design: Fraud Detection System

Key challenge: extreme class imbalance (fraud rate ~0.1-1%), adversarial (fraudsters adapt), need low latency (<100ms), high recall critical (miss as few frauds as possible).

Feature Engineering for Fraud

- **Transaction features:** Amount, merchant category, hour/day, location, device fingerprint
- **Behavioral features:** Velocity (transactions in last 1h/24h/7d), average spend pattern, distance from home location
- **Network features:** Graph of shared devices/IPs/cards. Fraud rings cluster in graph. Use GNN or graph stats (degree, clustering coeff)
- **Historical risk scores:** Past fraud flags, account age, failed auth attempts

Model Stack

33. **Real-time rules:** Immediate blocks for obvious fraud (amount > $50K in new country). No ML needed. Sub-millisecond.
34. **Real-time ML (<100ms):** XGBoost or LightGBM on pre-computed features from feature store. Outputs risk score.
35. **Near-real-time ML (<1s):** Deeper DNN with graph features. Higher recall but more computation.
36. **Batch review (async):** Complex patterns requiring historical analysis. Flag for human review.

Threshold tuning: Plot precision-recall curve. Business decision: what's the cost of false positive (frustrating good customer) vs false negative (missed fraud)? Choose threshold that minimizes total cost.

6.5 Design: Demand Forecasting (Amazon-Specific)

Amazon uses ML for inventory planning, logistics, pricing. Core problem: predict demand for millions of SKUs across thousands of locations, multiple time horizons (1-day to 6-month).

Feature Engineering for Time Series

- Lag features: demand at t-1, t-7, t-14, t-28, t-365 (same day last year)
- Rolling statistics: mean/std over last 7/14/28 days
- Calendar: day of week, month, holidays, Prime Day, Black Friday
- Promotion flags: is item on sale? Discount depth?

- Cross-product features: related items sold (cannibalization, substitution)

Model Options

DeepAR (Amazon)	Autoregressive RNN trained on all time series jointly. Shares statistical strength across items. Outputs probabilistic forecasts (distribution, not point estimate). Production at Amazon.
Temporal Fusion Transformer (TFT)	Multi-head attention + gating + LSTM. Handles multi-horizon forecasting, mixed input types, interpretable variable importances. SOTA on many benchmarks.
N-BEATS	Pure deep learning, no recurrence. Residual stacks with backward and forward forecasting. Very fast inference.

6.6 MLOps — Production ML Lifecycle

Model Registry & Versioning

MLflow, W&B, SageMaker Model Registry: track experiments (hyperparameters, metrics, artifacts). Version models. Promote versions through stages (Staging → Production → Archived). Enable rollback.

Data & Model Monitoring

PSI (Population Stability Index)	Measures distribution shift between reference and current: PSI = Σ (Act% - Exp%) * ln(Act%/Exp%). PSI < 0.1: no shift. 0.1-0.2: slight shift. > 0.2: major shift → retrain.
Prediction Drift	Monitor distribution of model output scores. Shift can indicate data quality issues before performance degrades (since we often don't have labels in real-time).
Shadow Mode	Run new model in parallel with production model, logging both outputs but only serving production. Validate new model behavior before exposing to users.
Canary Deployment	Route small % (e.g., 1%) of traffic to new model. Monitor closely. Gradually increase if stable.

```
  # MLflow Experiment Tracking
# MLflow experiment tracking
import mlflow
import mlflow.sklearn

with mlflow.start_run(run_name='xgboost_v2') as run:
    mlflow.log_params({'max_depth': 6, 'eta': 0.05, 'n_estimators': 300})
    model.fit(X_train, y_train)
    val_auc = roc_auc_score(y_val, model.predict_proba(X_val)[:, 1])
    mlflow.log_metric('val_auc', val_auc)
    mlflow.sklearn.log_model(model, 'model')
```

```
    print(f'Run ID: {run.info.run_id}')

# Register best model
mlflow.register_model(
    f'runs:/{best_run_id}/model',
    'fraud_detection_v2'
)
```

6.7 Model Compression & Efficient Inference

Quantization	Reduce weight precision from float32 → int8 or float16. 2-4x memory reduction, faster inference. Post-Training Quantization (PTQ): simple, no retraining. Quantization-Aware Training (QAT): simulate quantization during training, better accuracy.
Pruning	Remove weights with small magnitude (unstructured) or entire attention heads/neurons (structured). 50-90% of weights in large models are near-zero and removable. Requires fine-tuning after pruning.
Knowledge Distillation	Train small student model to mimic large teacher model. Student learns from soft probabilities (logits) of teacher. DistilBERT: 40% smaller, 60% faster, 97% of BERT performance.
ONNX Export	Open Neural Network Exchange. Convert PyTorch/TF model to ONNX format for optimized runtime inference (TensorRT, ONNX Runtime). Often 2-5x faster than native PyTorch.

PART 7: CODING, ALGORITHMS & ML IMPLEMENTATION

7. Coding, Data Structures & ML Implementation

7.1 Data Structures & Complexity — What You Must Know

Amazon/Microsoft AS interviews include LeetCode-style coding (Medium-Hard). You WILL be asked to implement ML algorithms efficiently. Understand Big-O well.

Complexity Quick Reference

```
# Complexity Reference
# Data structure time complexities
# List:        append O(1), insert/delete O(n), search O(n)
# Dict/Set:    get/set/delete O(1) avg, O(n) worst (hash collision)
# Heap:        push/pop O(log n), peek O(1)
# Sorted list: binary search O(log n), insert O(n)

# Sorting:
# QuickSort:  O(n log n) avg, O(n^2) worst, in-place
# MergeSort:  O(n log n) guaranteed, O(n) extra space, stable
# HeapSort:   O(n log n), O(1) space, not stable, cache-unfriendly
# TimSort:    Python's sort. O(n log n), O(n) space, stable, fast in practice
```

Implement K-Means from Scratch (Frequently Asked)

```
# K-Means++ from Scratch
import numpy as np

class KMeans:
    def __init__(self, k=3, max_iter=300, tol=1e-4, random_state=42):
        self.k = k; self.max_iter = max_iter
        self.tol = tol; self.rng = np.random.RandomState(random_state)

    def _init_centroids_plusplus(self, X):
        centroids = [X[self.rng.randint(len(X))]]
        for _ in range(self.k - 1):
            dists = np.min([np.sum((X - c)**2, axis=1) for c in centroids], axis=0)
            probs = dists / dists.sum()
            centroids.append(X[self.rng.choice(len(X), p=probs)])
        return np.array(centroids)

    def fit(self, X):
        self.centroids_ = self._init_centroids_plusplus(X)
        for i in range(self.max_iter):
            # Assign step
            dists = np.linalg.norm(X[:, None, :] - self.centroids_[None, :, :], axis=2)
            labels = dists.argmin(axis=1)
            # Update step
            new_centroids = np.array([X[labels == k].mean(axis=0) for k in range(self.k)])
            if np.linalg.norm(new_centroids - self.centroids_) < self.tol:
                break
            self.centroids_ = new_centroids
```

```
        self.labels_ = labels
        self.inertia_ = sum(np.sum((X[labels==k] - self.centroids_[k])**2) for k in
range(self.k))
        return self

    def predict(self, X):
        dists = np.linalg.norm(X[:, None, :] - self.centroids_[None, :, :], axis=2)
        return dists.argmin(axis=1)
```

Implement Logistic Regression from Scratch

```
  # Logistic Regression from Scratch with L2 Regularization
import numpy as np

class LogisticRegression:
    def __init__(self, lr=0.01, epochs=1000, l2=0.0):
        self.lr = lr; self.epochs = epochs; self.l2 = l2

    def _sigmoid(self, z): return 1 / (1 + np.exp(-np.clip(z, -250, 250)))

    def fit(self, X, y):
        n, d = X.shape
        self.w = np.zeros(d); self.b = 0.0
        for epoch in range(self.epochs):
            y_hat = self._sigmoid(X @ self.w + self.b)
            error = y_hat - y
            # Gradient with L2 regularization
            dw = (X.T @ error) / n + self.l2 * self.w
            db = error.mean()
            self.w -= self.lr * dw
            self.b -= self.lr * db
            if epoch % 100 == 0:
                loss = -np.mean(y*np.log(y_hat+1e-15) + (1-y)*np.log(1-y_hat+1e-15))
                print(f'Epoch {epoch}: Loss={loss:.4f}')

    def predict_proba(self, X): return self._sigmoid(X @ self.w + self.b)
    def predict(self, X, thresh=0.5): return (self.predict_proba(X) >= thresh).astype(int)
```

Implement a Transformer Attention Block

```
  # Multi-Head Attention Implementation
import torch, torch.nn as nn, torch.nn.functional as F, math

class MultiHeadAttention(nn.Module):
    def __init__(self, d_model=512, num_heads=8, dropout=0.1):
        super().__init__()
        assert d_model % num_heads == 0
        self.d_k = d_model // num_heads
        self.num_heads = num_heads
        self.W_q = nn.Linear(d_model, d_model)
        self.W_k = nn.Linear(d_model, d_model)
        self.W_v = nn.Linear(d_model, d_model)
        self.W_o = nn.Linear(d_model, d_model)
        self.dropout = nn.Dropout(dropout)

    def forward(self, q, k, v, mask=None):
        B, S, D = q.shape
        # Project and reshape to (B, heads, seq, d_k)
        Q = self.W_q(q).view(B, S, self.num_heads, self.d_k).transpose(1, 2)
        K = self.W_k(k).view(B, -1, self.num_heads, self.d_k).transpose(1, 2)
        V = self.W_v(v).view(B, -1, self.num_heads, self.d_k).transpose(1, 2)
        # Scaled dot-product attention
        scores = (Q @ K.transpose(-2, -1)) / math.sqrt(self.d_k)
```

```
        if mask is not None:
            scores = scores.masked_fill(mask == 0, -1e9)  # causal mask
        attn = self.dropout(F.softmax(scores, dim=-1))
        out = (attn @ V).transpose(1, 2).contiguous().view(B, S, D)
        return self.W_o(out)
```

7.2 Common LeetCode Patterns for AS Interviews

Top-K Elements (Heap)

```
  # Top-K Elements
import heapq

# Real-time top-K clicked items (Amazon interview question pattern)
class TopKStream:
    def __init__(self, k):
        self.k = k
        self.counts = {}
        self.heap = []  # min-heap of (count, item)

    def process(self, item):
        self.counts[item] = self.counts.get(item, 0) + 1
        # Rebuild heap periodically (or maintain sorted structure)

    def get_top_k(self):
        return heapq.nlargest(self.k, self.counts.items(), key=lambda x: x[1])

# O(n log k) solution for finding top-k frequent elements
def top_k_frequent(nums, k):
    counts = {}
    for n in nums: counts[n] = counts.get(n, 0) + 1
    return heapq.nlargest(k, counts.keys(), key=counts.get)
```

Sliding Window (Feature Engineering Pattern)

```
  # Sliding Window & Sequence Algorithms
# Rolling mean without library (efficient O(n))
def rolling_mean(data, window):
    result = []
    window_sum = sum(data[:window])
    result.append(window_sum / window)
    for i in range(window, len(data)):
        window_sum += data[i] - data[i - window]
        result.append(window_sum / window)
    return result

# Maximum subarray (Kadane's algorithm) -- sequence model baseline
def max_subarray(nums):
    max_sum = curr_sum = nums[0]
    for n in nums[1:]:
        curr_sum = max(n, curr_sum + n)
        max_sum = max(max_sum, curr_sum)
    return max_sum
```

Binary Search (Model Selection, Threshold Tuning)

```
  # Binary Search for Threshold Optimization
# Binary search for optimal decision threshold
def find_threshold(probs, labels, target_precision=0.95):
    lo, hi = 0.0, 1.0
    best_threshold = 0.5
```

```
    for _ in range(100):  # binary search iterations
        mid = (lo + hi) / 2
        preds = (probs >= mid).astype(int)
        tp = ((preds == 1) & (labels == 1)).sum()
        fp = ((preds == 1) & (labels == 0)).sum()
        precision = tp / (tp + fp + 1e-8)
        if precision >= target_precision:
            best_threshold = mid
            hi = mid  # can we push threshold higher?
        else:
            lo = mid
    return best_threshold
```

PART 8: ADVANCED TOPICS & SPECIALIZED AREAS

8. Advanced Topics — Expected at Senior Level

8.1 Graph Neural Networks (GNNs)

GNNs extend deep learning to graph-structured data. Each node aggregates information from its neighbors in a message passing framework.

```
h_v^(l+1) = σ(W * AGGREGATE({h_u^(l) : u ∈ N(v) ∪ {v}}))
```

Node v at layer l+1 aggregates embeddings of neighbors N(v). AGGREGATE can be sum, mean, max, or attention-weighted.

Key GNN Variants

GCN (Graph Convolutional Network)	Mean aggregation with degree normalization. Simple and effective. Limitation: can't distinguish different graph structures (e.g., nodes with same neighborhood pattern).
GraphSAGE	Samples a fixed-size neighborhood instead of using all neighbors. Inductive: can generalize to unseen nodes. Used at Pinterest (PinSage) for recommendations with billions of nodes.
GAT (Graph Attention Network)	Uses attention to weight neighbor contributions. Learns which neighbors matter. More expressive than mean aggregation.

Applications at Amazon/Microsoft: Social network analysis (fraud rings), knowledge graphs (entity linking), molecular property prediction, citation networks, product knowledge graphs.

8.2 Reinforcement Learning — Production Applications

Key RL Algorithms

Q-Learning	Model-free, off-policy. Learns Q(s,a) = expected cumulative reward. DQN: use neural network for Q-function. Experience replay + target network for stability. Used in game playing, robotics.
Policy Gradient (REINFORCE)	Directly optimize policy π(a|s). Update: ∇J(θ) = E[∇log π(a|s) * G_t]. High variance, mitigated with baseline (actor-critic). Used in NLP (sequence-to-sequence with RL reward).

PPO (Proximal Policy Optimization)	Clips the policy update to prevent too-large steps. Objective: min(r_t(θ)*A_t, clip(r_t(θ), 1-ε, 1+ε)*A_t). Stable and widely used. Foundation of RLHF for LLMs.

Multi-Armed Bandit	Simplified RL: no state transitions, just choose action and observe reward. Epsilon-greedy, UCB (Upper Confidence Bound), Thompson Sampling. Used for ad selection, recommendation exploration, clinical trials.

```
  # Thompson Sampling Multi-Armed Bandit
# Thompson Sampling for recommendation exploration
import numpy as np

class ThompsonSamplingBandit:
    def __init__(self, n_arms):
        # Beta distribution parameters for each arm
        self.alpha = np.ones(n_arms)   # successes + 1
        self.beta  = np.ones(n_arms)   # failures + 1

    def select_arm(self):
        # Sample from posterior Beta distribution
        samples = np.random.beta(self.alpha, self.beta)
        return samples.argmax()

    def update(self, arm, reward):
        self.alpha[arm] += reward
        self.beta[arm]  += (1 - reward)
```

8.3 Optimization Theory

Applied Scientists at Amazon work on optimization problems: supply chain, pricing, routing, scheduling. Know these fundamentals.

Convex Optimization	Minimizing a convex function over a convex set. Global optimum guaranteed. Includes LP, QP, SVM, LASSO. Tools: CVXPY, scipy.optimize.

Linear Programming (LP)	Minimize c^T x subject to Ax ≤ b, x ≥ 0. All constraints and objective linear. Solved efficiently by Simplex or Interior Point methods. Used for resource allocation, routing.

Integer Programming (IP)	LP with integer variables. NP-hard in general. Branch and bound, cutting planes. Used for bin packing, scheduling, network flow.

Lagrangian Relaxation	Incorporate constraints into objective with multipliers: L(x,λ) = f(x) + λg(x). Dual problem is always convex. Used to decompose large optimization problems.

```
  # Convex Optimization with CVXPY
import cvxpy as cp
import numpy as np

# Example: Portfolio optimization (minimize risk, target return)
n_assets = 10
```

```
mu = np.random.randn(n_assets) * 0.1 + 0.05  # expected returns
Sigma = np.random.randn(n_assets, n_assets)   # covariance matrix
Sigma = Sigma.T @ Sigma  # make positive semidefinite

w = cp.Variable(n_assets)
portfolio_risk   = cp.quad_form(w, Sigma)          # w^T Sigma w
portfolio_return = mu @ w

problem = cp.Problem(
    cp.Minimize(portfolio_risk),
    constraints=[
        portfolio_return >= 0.07,  # target return
        cp.sum(w) == 1,            # fully invested
        w >= 0                     # no short selling
    ]
)
problem.solve()
print(f'Optimal weights: {w.value}')
```

8.4 Time Series — Deep Dive

Classical Methods

ARIMA(p,d,q)	AutoRegressive Integrated Moving Average. p: AR order (past values), d: differencing (for stationarity), q: MA order (past errors). Fit with ACF/PACF plots. Box-Jenkins methodology.

Seasonal ARIMA (SARIMA)	Extends ARIMA with seasonal components (P,D,Q,m). m = season length. For monthly data with yearly seasonality: m=12.

Prophet (Facebook)	Additive model: trend + seasonality + holidays. Robust to outliers and missing data. User-friendly API. Good for business time series.

Test for Stationarity

Most time series models require stationarity (constant mean and variance). **ADF (Augmented Dickey-Fuller) test:** H0 = series has unit root (non-stationary). Low p-value → reject H0 → stationary. Fix non-stationarity with differencing, log transform, or detrending.

```
  # Time Series Stationarity & SARIMA
from statsmodels.tsa.stattools import adfuller
from statsmodels.tsa.statespace.sarimax import SARIMAX
import pandas as pd

# Stationarity test
result = adfuller(series, autolag='AIC')
print(f'ADF Statistic: {result[0]:.4f}, p-value: {result[1]:.4f}')
if result[1] < 0.05:
    print('Series is stationary')
else:
    print('Series is NOT stationary — apply differencing')
    series_diff = series.diff().dropna()

# SARIMA fitting
model = SARIMAX(series, order=(1,1,1), seasonal_order=(1,1,1,12))
result = model.fit(disp=False)
forecast = result.get_forecast(steps=12)
conf_int = forecast.conf_int()  # prediction intervals
```

8.5 Fairness, Explainability & Responsible AI

Fairness Definitions

Demographic Parity	P(ŷ=1|A=0) = P(ŷ=1|A=1). Equal positive prediction rates across groups. May conflict with accuracy.
Equalized Odds	TPR and FPR equal across groups. P(ŷ=1|y=1,A=0) = P(ŷ=1|y=1,A=1) and P(ŷ=1|y=0,A=0) = P(ŷ=1|y=0,A=1). Stricter than demographic parity.
Individual Fairness	Similar individuals receive similar predictions. Requires a similarity metric. Harder to enforce at scale.

Impossibility theorem: You cannot simultaneously satisfy demographic parity, equalized odds, AND calibration (except in degenerate cases). Business context determines which fairness criterion to optimize.

Explainability Methods

SHAP (SHapley)	Game-theoretic: each feature's contribution = average marginal contribution across all possible feature orderings. Consistent and locally accurate. TreeSHAP: efficient O(TL^2) algorithm for tree models.
LIME	Local Interpretable Model-agnostic Explanations. Fit a simple linear model in the local neighborhood of a prediction. Fast but less consistent than SHAP. Good for black-box models.
Integrated Gradients	For neural networks: attribute prediction to input features based on integral of gradients along path from baseline to input. Satisfies completeness: attributions sum to f(x) - f(baseline).

```
# SHAP Explanations
import shap

# SHAP for XGBoost (fast TreeSHAP)
explainer = shap.TreeExplainer(xgb_model)
shap_values = explainer.shap_values(X_val)

# Global feature importance
shap.summary_plot(shap_values, X_val, feature_names=feature_names)

# Local explanation for one prediction
shap.force_plot(explainer.expected_value, shap_values[0], X_val[0])

# SHAP for neural networks (slower, sampling-based)
explainer_nn = shap.GradientExplainer(model, X_train[:100])
shap_vals_nn = explainer_nn.shap_values(X_test[:10])
```

8.6 Distributed Training

Large models can't fit on a single GPU. Amazon/Microsoft train at massive scale.

Data Parallelism	Replicate model on N GPUs. Each GPU processes a different mini-batch. Aggregate gradients (AllReduce) across GPUs. PyTorch DDP. Scales well; GPU memory = single model.
Model Parallelism	Split model layers across GPUs. Layer k on GPU k. Sequential dependency limits efficiency. Pipeline parallelism (GPipe, PipeDream) uses microbatches to overlap computation.
Tensor Parallelism	Split individual layers (e.g., attention heads, FFN matrices) across GPUs. Used in Megatron-LM for transformers. Requires all-gather/reduce-scatter within each layer.
ZeRO (DeepSpeed)	Partitions optimizer states, gradients, and parameters across GPUs instead of replicating. ZeRO-3 partitions all three: reduces memory by ~8x with little communication overhead. Standard for LLM training.

PART 9: TOP 100 INTERVIEW QUESTIONS & EXPERT ANSWERS

9. Top 100 Questions — With Deep Answers

These reflect actual Amazon and Microsoft Applied Scientist interview questions. Answers are designed to be comprehensive — the kind you'd give in a 5-10 minute deep-dive discussion.

Q1. Derive the gradient of the MSE loss with respect to weights w.

MSE = $(1/N)||y - Xw||^2 = (1/N)(y - Xw)^T(y - Xw)$. Expanding: $(1/N)(y^Ty - 2w^TX^Ty + w^TX^TXw)$. Taking derivative w.r.t. w: $\partial L/\partial w = (1/N)(-2X^Ty + 2X^TXw) = (2/N)X^T(Xw - y) = (2/N)X^T(\hat{y} - y)$. At optimum (set to zero): $X^TXw = X^Ty \rightarrow w^* = (X^TX)^{-1}X^Ty$ (Normal Equation). Note: numerically prefer gradient descent or QR decomposition over explicit matrix inversion.

Q2. Derive the gradient of binary cross-entropy with sigmoid output.

Loss $L = -[y \log(\sigma(z)) + (1-y) \log(1-\sigma(z))]$. Let $\sigma = 1/(1+e^{-z})$. First note: $d\sigma/dz = \sigma(1-\sigma)$. Then: $\partial L/\partial z = -[y * (1/\sigma) * \sigma(1-\sigma) - (1-y) * (1/(1-\sigma)) * \sigma(1-\sigma)] = -[y(1-\sigma) - (1-y)\sigma] = \sigma - y = \hat{y} - y$. Elegantly: the gradient is simply the prediction error! This is why logistic regression with cross-entropy is so clean. For weight w: $\partial L/\partial w = (\hat{y} - y) * x$.

Q3. What is the curse of dimensionality and how does it affect distance-based models?

In high dimensions ($d >> 1$), two key phenomena occur: (1) Volume concentration: most of the volume of a hypersphere is concentrated in a thin shell near the surface. (2) Distance concentration: distances between random points become nearly equal — the ratio of max to min distance approaches 1. This means KNN, K-Means, and SVM with RBF kernel degrade badly in high-d. Solutions: PCA (reduce to intrinsic dimension), feature selection, metric learning (learn a good distance function), embeddings. Rule of thumb: need $\sim 2^d$ samples to maintain constant density as d grows.

Q4. Explain eigenvalue decomposition and how it relates to PCA.

For symmetric matrix A: $A = Q\Lambda Q^T$ where Q has eigenvectors as columns, Λ is diagonal with eigenvalues. Covariance matrix $C = (1/N)X^TX$ is symmetric positive semidefinite — always has real, non-negative eigenvalues. PCA: decompose $C = V\Lambda V^T$. Columns of V are principal components (directions of maximum variance). Eigenvalue λ_i = variance explained by PC i. Project: $X_{pca} = XV$. Keep top-k PCs explaining 95% of variance: k/total_components ratio. PCA via SVD: $X = U\Sigma V^T$. Same PCs as eigendecomposition of X^TX but more numerically stable.

Q5. Explain the Central Limit Theorem and its implications for ML.

CLT: Let $X_1,\ldots,X_n$ be i.i.d. with mean μ and variance σ^2. Then $(\bar{X} - \mu) / (\sigma/\sqrt{n}) \to N(0,1)$ as $n \to \infty$, regardless of the original distribution. Implications for ML: (1) Confidence intervals are valid for any distribution with $n > 30$ or so. (2) Mini-batch gradient estimates are approximately normal around true gradient — justifies analyzing training dynamics. (3) Bootstrap works because of CLT. (4) Parameter estimates (MLE) are asymptotically normal — enables confidence intervals for model parameters.

Q6. How do you design a proper A/B test?

1) Define hypothesis: H0 = no difference in metric M between control A and treatment B. 2) Choose primary metric (e.g., conversion rate, revenue per user) and guardrail metrics (latency, error rate — must not degrade). 3) Power analysis: compute required sample size $n = (z_{\alpha/2} + z_\beta)^2 * 2\sigma^2 / \delta^2$ where δ = MDE. For proportions, use proportion_effectsize. 4) Randomization: hash user_id to ensure stable assignment. Use holdout (not time-based split — avoids novelty effect). 5) Run for at least 2 business cycles (2 weeks minimum for daily seasonality). 6) Check for SRM (Sample Ratio Mismatch): verify treatment/control split is as designed. 7) Analyze with appropriate test (t-test for means, chi-squared for proportions). 8) Report effect size + confidence interval, not just p-value. 9) Consider multiple testing correction (Bonferroni, Benjamini-Hochberg) if testing multiple metrics.

Q7. What is Simpson's Paradox and give a ML-specific example.

Simpson's Paradox: a trend that appears in several groups reverses when the groups are combined. ML example: Suppose Model A has higher overall accuracy than Model B. But when you stratify by 'young' and 'old' age groups, Model B outperforms Model A in BOTH groups. How? Model A was trained/tested on more 'old' users (who are easier to classify), inflating its overall accuracy. This is common in evaluation of models with class imbalance or demographic skew. The lesson: always segment evaluation by relevant subgroups. A model that 'improves' overall may be getting worse for underrepresented groups. Use stratified evaluation and fairness metrics.

Q8. What is MLE vs MAP and when would you use MAP?

MLE: find θ that maximizes $P(D|\theta)$. MAP: find θ that maximizes $P(\theta|D) = P(D|\theta)*P(\theta)/P(D)$, i.e., maximizes $P(D|\theta)*P(\theta)$. MAP = MLE + prior. MLE can overfit with small data. MAP regularizes by incorporating prior beliefs. Example: Ridge regression = MAP with Gaussian prior on weights. Lasso = MAP with Laplace prior. Bayesian ML goes further: computes full posterior distribution $P(\theta|D)$, enabling uncertainty estimates. Use MAP when: you have domain knowledge (prior), small dataset, or want a principled form of regularization.

Q9. Walk through the complete bias-variance decomposition.

For regression with squared loss: $E[(y - \hat{f}(x))^2] = \text{Bias}(\hat{f}(x))^2 + \text{Var}(\hat{f}(x)) + \sigma^2$ where: Bias = $E[\hat{f}(x)] - f(x)$ (systematic error from wrong model assumptions), Var = $E[(\hat{f}(x) - E[\hat{f}(x)])^2]$ (sensitivity to training data variation), σ^2 = irreducible noise. Practical implications: K-NN with K=1 has zero bias but high variance (memorizes training data). K-NN with K=N predicts constant (high bias, zero variance). Ridge regression reduces variance at cost of bias. Deep networks: overparameterized models can have surprisingly low bias AND low variance (double descent phenomenon).

Q10. Explain and derive the VC dimension and its relation to generalization.

VC dimension (Vapnik-Chervonenkis): the largest set of points that a hypothesis class H can shatter (classify in all 2^n possible ways). Examples: linear classifiers in d dimensions → VC dim = d+1.

Decision stumps → VC dim = 1. k-NN → VC dim = infinity. Generalization bound: with probability ≥ 1-δ, true error ≤ training error + O(sqrt(VCdim * log(N) / N)). Practical implication: models with higher VC dimension (more complex) need exponentially more data to generalize. This is the theoretical foundation for why regularization works.

Q11. How does gradient boosting work mathematically?

GBM minimizes loss L iteratively. At step m: find $h_m = \text{argmin}_h \Sigma L(y_i, F_{m-1}(x_i) + h(x_i))$. This is hard to solve generally. Key insight: fit h_m to the negative gradient (pseudo-residuals) $r_{im} = -[\partial L(y_i, F(x_i))/\partial F(x_i)]$ at $F = F_{m-1}$. Update: $F_m = F_{m-1} + \eta * h_m$. For MSE: residuals = $y - \hat{y}$ (actual residuals). For log-loss: $r_i = y_i - \sigma(F_{m-1}(x_i))$ (same as logistic regression gradient). For XGBoost: use second-order Taylor approximation $L \approx [g_i*h(x_i) + (1/2)*H_i*h(x_i)^2]$ where g_i = first derivative, H_i = second derivative. Second-order info enables optimal leaf weights in closed form.

Q12. What is the kernel trick and why does it work?

The kernel trick lets us implicitly compute dot products in a high-dimensional (or infinite-dimensional) feature space $\varphi(x)$ without ever explicitly computing $\varphi(x)$. Kernel: $K(x, z) = \varphi(x)\cdot\varphi(z)$. Example: polynomial kernel $K(x,z) = (x\cdot z + c)^d$ corresponds to φ that maps d features to $O(d^n)$ features — exponential blowup! But $K(x,z)$ is computable in $O(d)$ time. SVM dual formulation only requires dot products between data points (not individual coordinates), enabling kernel substitution. RBF kernel $K(x,z) = \exp(-\gamma||x-z||^2)$ corresponds to infinite-dimensional feature space. Mercer's theorem: any positive semi-definite function is a valid kernel.

Q13. How does backpropagation work? Walk through a complete example.

Backprop applies the chain rule recursively from output to input. Example: $L = \text{MSE}(\text{relu}(XW1 + b1)W2 + b2, y)$. Forward pass: $z1 = XW1+b1$, $a1 = \text{relu}(z1)$, $z2 = a1\ W2 + b2$, $L = (1/2N)||z2-y||^2$. Backward pass: $\partial L/\partial z2 = (z2-y)/N$. $\partial L/\partial W2 = a1^T @ \partial L/\partial z2$. $\partial L/\partial a1 = \partial L/\partial z2 @ W2^T$. $\partial L/\partial z1 = \partial L/\partial a1 * I(z1>0)$ [relu derivative]. $\partial L/\partial W1 = X^T @ \partial L/\partial z1$. $\partial L/\partial b2 = \text{sum}(\partial L/\partial z2)$. $\partial L/\partial b1 = \text{sum}(\partial L/\partial z1)$. Key: autograd systems (PyTorch, JAX) build a computation graph during forward pass and traverse it in reverse during backward pass.

Q14. Explain the vanishing gradient problem and all solutions.

Vanishing gradient: in deep networks with sigmoid/tanh, gradients shrink exponentially backward. $|d(\text{sigmoid})/dx| \leq 0.25$ at maximum. After 10 layers, gradient magnitude $\rightarrow (0.25)^{10} \approx 10^{-6}$. Early layers receive near-zero updates. Solutions: (1) ReLU activations: gradient = 1 for $x>0$, no saturation. (2) Residual connections (ResNet): identity shortcuts let gradients flow directly from output to any layer. (3) Batch normalization: normalizes activations, keeping them in non-saturating range. (4) LSTM gates: multiplicative gating controls gradient flow, allowing long-range dependencies. (5) Gradient clipping: prevents very small gradients from having wrong sign after floating point ops. (6) Better initialization (He/Xavier): keeps initial gradient magnitudes reasonable.

Q15. What is self-attention and why is it $O(N^2)$?

Self-attention computes pairwise interactions between all token pairs. For sequence of length N: Q, K, V each have shape (N, d_k). Attention matrix = QK^T has shape (N, N) — every token attends to every other token. This is the $O(N^2)$ step. Memory: storing the $N\times N$ attention matrix requires $O(N^2)$ space. For $N=4096$, this is 16M values per head, per layer. Solutions to quadratic complexity: (1) Sparse attention: each token attends to only a fixed window + global tokens (Longformer, BigBird). (2) Linear attention: approximate softmax with kernel trick $\rightarrow O(N)$. (3) Flash Attention: reorder computation to use SRAM tiles, same math but $O(N)$ memory. (4) Sliding window attention: only attend to local context.

Q16. Compare BERT, GPT, and T5. When would you use each?

BERT: encoder-only, bidirectional, masked LM pre-training. Strong contextual representations. Best for: text classification, NER, extractive QA, sentence similarity. Cannot generate text. GPT: decoder-only, causal (unidirectional), causal LM pre-training. Generates text autoregressively. Best for: text generation, completion, dialogue, summarization, in-context learning (few-shot). T5 (Text-to-Text Transfer Transformer): encoder-decoder. Frames ALL tasks as text-to-text (even classification: input='classify: positive or negative: ...', output='positive'). Versatile. Best for: tasks that can be framed as seq2seq: translation, summarization, Q&A, classification. Use BERT when you have labeled data and need a strong encoder. Use GPT/LLaMA when you need generation. Use T5 when you want a unified approach across tasks.

Q17. How does LoRA work and when should you use it over full fine-tuning?

LoRA (Low-Rank Adaptation): instead of updating the full weight matrix $W \in R^{d\times k}$, we add a low-rank decomposition $\Delta W = BA$ where $B \in R^{d\times r}$ and $A \in R^{r\times k}$ with $r << \min(d,k)$. Only A and B are trained (randomly initialized with B=0 so ΔW starts at zero). At inference: W + BA is merged back, zero overhead. Why it works: the intrinsic dimensionality of fine-tuning is much lower than the parameter count suggests. Rank r=8 often works for most tasks. Use LoRA when: large base model (7B+ params), limited GPU memory, multiple fine-tuned variants needed (swap A,B weights per task), or when full fine-tuning would overfit (small dataset). Use full fine-tuning when: maximum quality is needed, large high-quality dataset available, compute is not a constraint.

Q18. What are Flash Attention and GQA, and how do they improve transformer efficiency?

Flash Attention: computes exact attention without materializing the full N×N attention matrix. Uses tiling: load Q, K, V tiles into fast SRAM, compute attention for that tile, accumulate output. Result: $O(N)$ memory instead of $O(N^2)$. 2-4x wall-clock speedup because bandwidth (not FLOPS) is the bottleneck. Flash Attention 2/3 further optimizes CUDA implementation. GQA (Grouped Query Attention): instead of h query heads each with their own K,V, groups of query heads share K,V heads. Reduces KV cache size during inference (critical for long context). MHA: h Q heads, h K heads, h V heads. MQA (Multi-Query): h Q heads, 1 K head, 1 V head. GQA: h Q heads, g K/V heads where $1 < g < h$. LLaMA-2, Mistral use GQA.

Q19. Design a real-time fraud detection system for Amazon at scale (100K TPS).

Requirements: 100K transactions/second, latency < 50ms P99, precision and recall both > 95%. Architecture: (1) Feature extraction: pre-computed user features (aggregated hourly via streaming Spark/Kinesis) stored in Redis feature store. Real-time features: transaction amount, time delta, device hash. (2) Rule engine (first layer, < 1ms): block obvious fraud (large amount from new device in foreign country). (3) ML model (XGBoost served via TorchServe, ~5ms): 200-300 features including velocity features (tx count last 1h, 24h), graph features (device/IP cluster). (4) Threshold: tune for high recall (FN = missed fraud = expensive). (5) Async review: suspected but not blocked transactions sent to async queue for human review. (6) Model updates: retrain weekly on new fraud patterns, online feature distribution monitoring for drift.

Q20. How would you build a search system for product queries at Amazon?

Two-stage: Stage 1 — Retrieval: (a) Lexical: BM25 over product title/description/attributes using Elasticsearch/OpenSearch. (b) Semantic: bi-encoder (BERT-based) to get product and query embeddings, ANN search via FAISS. Merge with RRF (Reciprocal Rank Fusion). Stage 2 — Ranking:

LambdaRank model with features: query-product semantic similarity, click/purchase history, product popularity, price, availability, customer ratings, recency. Training signal: human relevance judgments + click logs (position-debiased). Optimization: NDCG@10. Online: A/B test ranking changes. Monitoring: query success rate (click in top 3), abandonment rate.

Q21. How would you design a personalized news feed ranking system?

Problem: rank N candidates (500+) for user in < 100ms. Data: user's past interactions (clicks, reads, shares), content metadata (topic, author, freshness). Architecture: (1) Candidate retrieval: collaborative filtering + content-based. (2) Features: user embedding (from history), content embedding (BERT title), user-content cross features, freshness decay, author follow status. (3) Model: DCN v2 (Deep & Cross Network) or DLRM. Loss: multi-task — optimize click + read time + share (weighted). (4) Diversity: MMR post-processing to avoid filter bubble. (5) Freshness: decay score based on article age. (6) Exploration: 5% epsilon-greedy for new content. Online metric: session engagement, 7-day retention.

Q22. A model's offline AUC improved from 0.82 to 0.87, but online CTR decreased by 2%. How do you debug this?

This is a classic train-serve skew / metric mismatch problem. Investigation steps: (1) Check feature pipeline: are serving features computed identically to training? Common bug: training uses 'future-aware' features (leakage). (2) Check distribution shift: compare feature distributions between training data and live traffic. PSI > 0.2 for any feature? (3) Check label quality: was the new model trained on different labels (e.g., different time period with different user behavior)? (4) Check metric alignment: offline AUC measures ranking quality of past data; CTR measures click rate on live traffic. These can diverge if offline data has selection bias (only logged shown items). (5) Segment analysis: which user segments drove the CTR decline? New users? Mobile? Specific regions? (6) Counterfactual: use importance weighting or counterfactual evaluation to better align offline metric with online. (7) Check A/B test validity: SRM? Novelty effect? Run longer.

Q23. Implement K-Nearest Neighbors classifier with k-d tree from scratch.

KNN stores all training data. For each test point: find k nearest neighbors (by Euclidean distance) and take majority vote. Naive: O(Nd) per query. k-d tree: recursively partition feature space along axis of maximum variance. Average O(k log N) per query. Build: O(N log N). Falls apart in high dimensions (d > 20) — k-d tree degrades to O(N). Better alternatives in high-d: ball tree, LSH (Locality Sensitive Hashing), or approximate methods. Note: always normalize features before KNN — different scales will dominate distance calculation.

Q24. Implement efficient online mean and variance computation (Welford's algorithm).

Welford's online algorithm computes running mean and variance in O(1) per update, O(N) total, with numerical stability. count += 1; delta = x - mean; mean += delta / count; delta2 = x - mean; M2 += delta * delta2; variance = M2 / (count - 1). Why does this matter? For streaming ML feature standardization, you can't store all data. Also useful for computing per-batch statistics. Compare to naive approach: sum all values then divide — suffers from catastrophic cancellation for large values with small variance.

Q25. How would you implement a streaming top-K algorithm?

Use a min-heap of size K. For each new element: if heap has < K elements, push. Else if element > heap[0] (minimum), pop minimum and push new element. After N elements, heap contains top K. Time: O(N log K). Space: O(K). For approximate top-K on massive streams: Count-Min Sketch (estimates frequency with small error), Space-Saving algorithm (exact top-K with bounded memory). For distributed top-K: each worker maintains local top-K, merge all and take global top-K.

Q26. Implement a decision tree classifier in Python.

Key steps: (1) At each node, iterate over all features and all unique split points, compute Gini impurity for each split. (2) Choose split with lowest weighted Gini. (3) Recursively build left and right subtrees. (4) Stop if: max_depth reached, min_samples_leaf constraint, or node is already pure. Prediction: traverse tree based on feature values, return majority class at leaf. Space: O(2^max_depth). Inference: O(max_depth). Key observation: decision trees are trained greedily (locally optimal splits), not globally. This is why ensembles (Random Forest, XGBoost) are much more powerful.

Q27. You have 1M labeled samples but only 1K have high-quality labels. How do you train?

This is a semi-supervised learning problem. Options: (1) Pre-train on 1M (with noisy labels or self-supervised) → fine-tune on 1K. Self-supervised: masked autoencoders, contrastive learning (SimCLR). (2) Pseudo-labeling / self-training: train on 1K, predict on unlabeled, add high-confidence predictions as pseudo-labels, retrain. Iterate. (3) Label propagation: assume nearby points (in feature space) have similar labels. Propagate from labeled to unlabeled. (4) Active learning: use current model's uncertainty to select the MOST informative unlabeled samples for human labeling. Maximize information gain per labeling dollar. (5) Data augmentation: dramatically expand 1K high-quality samples with augmentations (flip, crop, noise, paraphrase for NLP). Quality > quantity for fine-tuning.

Q28. Your model works well in testing but degrades after 2 weeks in production. What's happening and how do you fix it?

Classic concept drift / data drift scenario. Diagnosis: (1) Monitor feature distributions (PSI test). Is input distribution shifting? (2) Monitor prediction distribution. Is model outputting different scores? (3) If labels are available with delay, track actual performance metrics. (4) Check external factors: seasonality? business change? upstream data pipeline change? Solutions: (1) Data drift: retrain on recent data (sliding window), OR add recent data to training set. (2) Concept drift: periodic retraining (weekly/monthly), OR online learning (update model continuously). (3) Detection: set up automated PSI monitoring with alerting. (4) Prevention: feature engineering that's robust to drift (e.g., relative rather than absolute features).

Q29. How would you handle training a model when positive examples are 1 in 10,000?

Extreme imbalance (fraud, rare disease, etc.). Multiple strategies: (1) Change loss function: focal loss (reduces relative loss for well-classified easy examples, focuses on hard examples): $FL = -\alpha(1-pt)^\gamma \log(pt)$. (2) Class-weighted loss: weight positive class by 10000x. (3) Undersampling majority + oversampling minority (SMOTE). But with 1:10000 ratio, even 100x oversampling still leaves 1:100. (4) Anomaly detection reframing: treat positive class as anomalies, train one-class model on negatives. (5) Data collection: worth investing in getting more positive examples. (6) Evaluation: use PR-AUC (precision-recall curve), not ROC-AUC — ROC is misleading with extreme imbalance. Report at specific operating points (recall@precision=90%).

Q30. How would you evaluate whether your LLM-based application is production-ready?

Multi-dimensional evaluation: (1) Automated metrics: BLEU/ROUGE for generation, accuracy for classification, perplexity. LLM-as-judge: use a stronger LLM to evaluate quality. (2) Human evaluation: preference studies (A vs B), Likert scale ratings, task completion rate. (3) Safety: hallucination rate (is the model making things up?), harmful content detection, PII leakage. (4) Robustness: adversarial inputs, prompt injection attacks, out-of-distribution inputs. (5) Latency and cost: P50/P99 latency, cost

per 1000 queries. (6) Faithfulness for RAG: does the answer stay grounded in retrieved context? RAGAS framework. (7) Shadow deployment: run in parallel with existing system. Compare on real traffic. (8) Gradual rollout: 1% → 5% → 20% → 100%. Monitor business metrics at each stage.

Q31. How would you improve Alexa's entity recognition for multilingual users?

Challenge: user switches languages mid-sentence (code-switching), accents, domain-specific entities (Amazon product names). Approach: (1) Model: multilingual BERT (mBERT) or XLM-R as backbone — trained on 100+ languages. (2) Fine-tune on NER with code-switched training data. (3) Entity linking: map extracted mentions to Amazon product catalog using dense retrieval (bi-encoder) + knowledge graph. (4) Language detection: lightweight LID model runs first to determine language(s) and inform tokenization. (5) User-specific adaptation: if user has purchase history, weight their known products higher. (6) Evaluation: exact match, partial match, F1 by entity type and language. (7) Data: collect misrecognition logs, annotate, add to training. Active learning for most common failures.

Q32. Amazon's recommendation CTR dropped 15% after a model update. Walk through your investigation.

Systematic approach: (1) Confirm the drop: is it in A/B test or a full rollout? Check sample ratio mismatch first. (2) Segment the drop: which surface? (homepage, product page, email). Which user segment? (new vs returning, mobile vs desktop). Which item categories? (3) Check model outputs: did score distribution change? Are we recommending less popular/less relevant items? (4) Check feature pipeline: any feature missing or corrupted? Null rates? Distribution shift? (5) Check training data: was training data from a different time period? Any label leakage in old model that was removed? (6) Compare top-recommended items: what are the top-10 recommendations for typical users? Do they make sense? (7) Rollback immediately if drop is real. (8) Retrain with controlled changes (one change at a time) to isolate root cause.

Q33. Design an ML system for Copilot code completion in VS Code.

Requirements: < 100ms latency, high relevance, handles 10M developers. (1) Model: Base = fine-tuned code LLM (Codex/GPT-4 architecture). (2) Context: current file, open files (cross-file context), project type. (3) Retrieval: RAG over project codebase — embed all functions/classes, retrieve relevant context with FAISS. (4) Personalization: user's coding style (variable naming, patterns) via lightweight LoRA adapter per user. (5) Serving: speculative decoding (small draft model predicts tokens, large model verifies) for latency. Batching requests across users. (6) Evaluation: acceptance rate (did user accept suggestion?), edit distance between suggestion and final code, task completion time. (7) Safety: filter PII, secrets (API keys), copyrighted code patterns.

Q34. How would you build the ML components for Microsoft's email priority system (Focused Inbox)?

Problem: binary classification — is this email 'focused' (important) or 'other'? Features: sender relationship (previous replies, contact frequency), content features (BERT embeddings of subject/body), time-of-day, organizational features (manager, direct reports), explicit user feedback (moved to/from focused). Model: two-stage — fast features (sender score) for real-time, deeper features (content) for async. Personalization: per-user preference learning — explicitly model each user's engagement patterns. Cold start: use organization-wide patterns until enough personal data. Privacy: all user-specific models trained on-device (federated learning) to avoid sending email content to server. Evaluation: click-through on focused emails, email response rate, user override rate (moved out of focused).

Q35. Tell me about a time you had to make a decision with incomplete data. (Amazon LP: Are Right a Lot)

Structure your answer with STAR (Situation, Task, Action, Result). Key elements: (1) Acknowledge the uncertainty explicitly — show you understand what data was missing. (2) Describe how you estimated using proxies, analogies to similar situations, or confidence intervals. (3) Show you made a decision anyway — 'bias for action' at Amazon. (4) Describe guardrails you put in place to catch mistakes quickly. (5) Quantify the result. Example pattern: 'We needed to decide whether to A/B test a new recommendation algorithm before we had enough labeled data. I used a combination of offline proxy metrics and a small-scale pilot to estimate the expected lift, set a clear kill switch criterion, launched to 1% of traffic, and monitored closely. The model ultimately increased CTR by 3%.

Q36. Describe a project where you drove technical innovation. (Amazon LP: Invent and Simplify)

Show you identified an inefficiency, designed a novel solution, built it, and measured impact. Key elements: (1) Context: describe the specific business problem. (2) Existing approach and its limitations (complexity, cost, accuracy). (3) Your novel idea — what made it different? (4) How you convinced stakeholders. (5) Implementation challenges and how you overcame them. (6) Quantified impact (latency improvement, accuracy gain, cost reduction). Amazon values 'simplify' as much as 'invent' — mention if you reduced complexity while improving performance.

Q37. How do you handle high-cardinality categorical features (e.g., user_id with 100M unique values)?

High cardinality causes problems with one-hot encoding (100M columns!) and target encoding (overfitting). Options: (1) Embedding layer: learn a dense d-dimensional vector per category end-to-end with the model (like word embeddings). Most powerful. Requires deep learning. (2) Hashing trick: map categories to fixed-size hash table. Collisions acceptable. Used in LLMs for large vocabularies. (3) Target encoding with regularization: blend category mean with global mean, especially for rare categories. Use cross-validation to prevent leakage. (4) Frequency encoding: replace category with count/frequency. Simple, doesn't cause curse of dimensionality. (5) Grouping: cluster rare categories into 'Other'. (6) For user_id specifically: use pre-trained user embeddings from collaborative filtering.

Q38. How do you prevent data leakage in time-series feature engineering?

Time series leakage: using future information to predict the past. Common bugs: (1) Computing global statistics (mean, std) on full dataset then using them in training — test data 'leaked' into training normalization. Fix: fit scaler only on train split. (2) Lag features not correctly aligned — target at time t uses feature at time t+1. Fix: always verify lag direction with a simple sanity check. (3) Train/val split is random instead of time-based — model sees future patterns. Fix: always use time-ordered split. (4) Target encoding computed on full dataset — future target leaks into encoding. Fix: use cross-validation for target encoding. (5) Rolling features using future values. Fix: use .shift(1) before computing rolling stats in pandas. Code review practice: always visualize feature correlation with target — suspiciously high correlation is a leakage red flag.

Q39. What is the difference between L1 and L2 regularization and when to choose each?

L1 (Lasso): adds $\lambda\Sigma|w_i|$ to loss. Gradient: λ*sign(w_i). This has a constant magnitude regardless of $|w_i|$, which continuously drives small weights to exactly zero. Produces sparse solutions — great for feature selection. Geometric interpretation: L1 ball has corners aligned with axes; solution tends to hit corners (zero coordinates). L2 (Ridge): adds $\lambda\Sigma w_i^2$ to loss. Gradient: $2\lambda w_i$. Drives weights toward zero proportionally. Never exactly zero (in practice). Keeps all features but with small weights. Better

when all features are somewhat useful. Elastic Net: $\lambda 1 * \Sigma |w_i| + \lambda 2 * \Sigma w_i^2$. Best of both. Sparsity + handles correlated features better than pure L1 (L1 tends to pick one of correlated features arbitrarily). From Bayesian perspective: L1 = Laplace prior (sharp peak at 0), L2 = Gaussian prior (soft at 0).

Q40. Explain the concept of attention from first principles. Why does scaling by sqrt(d_k) help?

Attention asks: 'how much should I weight each value V based on relevance to my query Q against keys K?' The dot product Q·K measures alignment. Without scaling, for large d_k, dot products grow as $O(d_k)$ in magnitude because they sum d_k terms. Large dot product magnitudes push softmax into saturated regions (one entry near 1, rest near 0). In saturation: gradient of softmax ≈ 0 — vanishing gradients during training. Scaling by $1/\sqrt{d_k}$ ensures dot product variance ≈ 1 (assuming Q and K have unit variance): $Var(Q \cdot K) = Var(\Sigma_j q_j * k_j) = d_k * Var(q_j) * Var(k_j) \approx d_k$. Dividing by $\sqrt{d_k}$ normalizes variance to 1. This keeps softmax in a well-behaved regime, enabling stable training.

Q41. How does the Transformer handle variable-length inputs and outputs?

Encoder: pads inputs to same length in a batch. Attention mask marks padding positions — those positions get -∞ before softmax so they contribute zero to attention. Decoder (autoregressive generation): starts with start token. At each step, generates one token and appends to sequence. Causal mask prevents each position from attending to future positions (upper triangle in attention matrix = -∞). Positional encoding gives the model order information since attention itself is permutation-invariant. For very long inputs: truncate and stride (sliding window), or use hierarchical encoding (chunk + summarize). Beam search: maintain B best partial sequences during decoding for better quality.

Q42. What are the trade-offs between batch size and model performance?

Large batch: (1) More efficient GPU utilization — better parallelism. (2) More accurate gradient estimate — less noise. (3) Often requires higher learning rate (linear scaling rule: LR ∝ batch_size) or warmup. (4) May converge to 'sharp minima' that generalize worse (sharp minima = narrow valleys in loss landscape — small perturbations cause large loss increase). Small batch: (1) Noisy gradients provide implicit regularization — helps escape sharp minima, finds flat minima. (2) More gradient steps per epoch — can converge faster in wall-clock time for small datasets. (3) Essential for some training paradigms (online learning). Practical guidance: start with batch=32-256. Scale up LR linearly with batch. For fine-tuning LLMs: small batches (4-16) with gradient accumulation often work best.

Q43. Explain NDCG and why it's preferred over accuracy for recommendation/ranking.

Accuracy treats all items equally: getting item 1 or item 1000 wrong is equally bad. For ranking, this is wrong — errors at the top are much more costly than at the bottom (users rarely scroll). NDCG penalizes relevant items ranked lower logarithmically: DCG@K = Σ_{i=1}^K (2^rel_i - 1) / log2(i+1). Position 1 gives full gain; position 2 gives log2(2)=1, so half gain; position 3 gives log2(3)≈0.58 gain, etc. NDCG normalizes DCG by ideal DCG (perfect ranking). NDCG@10 measures quality of top 10. Other ranking metrics: MRR (Mean Reciprocal Rank) — 1/rank of first relevant item; MAP (Mean Average Precision) — precision at each rank where a relevant item appears. For A/B testing ranking: NDCG is better than CTR because it accounts for position bias.

Q44. How do you perform hyperparameter tuning at scale?

Options in increasing sophistication: (1) Manual: educated guesses + intuition. Doesn't scale. (2) Grid search: exhaustive over predefined grid. O(k^n) experiments. Only for < 3 hyperparameters. (3) Random search: sample random combinations. 60× more efficient than grid for 5+ hyperparameters (Bergstra & Bengio, 2012). Each hyperparameter gets effective unique values, not wasted. (4) Bayesian optimization (Optuna, HyperOpt): model the surrogate function HP → validation metric. Use acquisition function (Expected Improvement) to select next point. Efficient: 3-5x fewer trials than random. (5) Population-based training (PBT): evolve a population of models. Good for dynamic LR schedules. (6) Neural Architecture Search (NAS): automated architecture design. Very expensive. Best practice: use Optuna with 100-200 trials. Log all experiments in MLflow. ALWAYS tune on validation set, report final metric on test set (no peeking!).

Q45. What is knowledge distillation and how do you implement it?

Knowledge distillation: train a small student model to match the output distribution (logits) of a large teacher model, not just the hard labels. The 'dark knowledge' in soft probabilities contains class relationships (e.g., teacher's P(cat)=0.6, P(dog)=0.3 is more informative than hard label 'cat'). Loss: L = α * CE(student_logits/T, hard_labels) + (1-α) * KL(softmax(teacher_logits/T),

softmax(student_logits/T)) * T^2. Temperature T > 1 softens distributions, revealing more information. Typical T=3-5, α=0.1-0.3. DistilBERT uses distillation from BERT: 6 layers vs 12, 40% smaller, retains 97% performance. Implementation: load teacher (frozen), train student with combined loss.

Q46. How would you detect and handle feature drift in production?

Detection methods: (1) Statistical tests: KS test (two-sample distribution test), Jensen-Shannon divergence, PSI (Population Stability Index). Run daily/hourly on each input feature. Alert if PSI > 0.2. (2) Monitoring dashboards: track percentiles (P25, P50, P75, P99) and null rates for all features over time. (3) Prediction drift: even if inputs look fine, monitor output score distribution. (4) Business metrics: sudden changes in downstream KPIs (CTR, revenue) may indicate drift. Response: (1) Minor drift: retrain on sliding window of recent data. (2) Major drift: investigate data pipeline first (upstream bug?). If real drift, may need to redesign features or add new features. (3) Covariate shift correction: importance weighting — weight training samples by P(x|test)/P(x|train). (4) Adaptive models: online learning algorithms update continuously.

Q47. What are embedding models and how do you evaluate them?

Embedding models encode inputs (text, images, users, items) into dense vector representations where semantic/behavioral similarity corresponds to geometric proximity. Training: contrastive learning (SimCLR, CLIP), triplet loss (anchor + positive close, anchor + negative far), or pair-wise (MSE on similarity scores). Evaluation: (1) Intrinsic: cosine similarity on labeled pairs (STS benchmarks for text). (2) Retrieval: Recall@K on nearest neighbor retrieval tasks. (3) Downstream: fine-tune on classification task — better embeddings = better fine-tuned accuracy. (4) Clustering quality: silhouette score on human-labeled clusters. (5) ANN efficiency: build-time, query-time, recall-precision trade-off at different probe counts. Key design choices: dimension (256-1536), normalization (L2 normalize for cosine similarity), negative mining strategy (hard negatives vs easy).

Q48. How does the PageRank algorithm work and what ML connections does it have?

PageRank: $PR(i) = (1-d)/N + d * \Sigma_{j\to i} PR(j)/out_degree(j)$. d = damping factor (0.85). Iterative computation until convergence. Can be expressed as: $PR = d * A * PR + (1-d)/N * 1$ where A is column-normalized adjacency matrix. This is a power iteration method computing the principal eigenvector of A. ML connections: (1) Graph Neural Networks use similar neighborhood aggregation. (2) Attention mechanism: attention weights are soft PageRank-like scores. (3) Matrix factorization: closely related to spectral methods on graphs. (4) Used in web search ranking, academic citation networks, biological network analysis. Personalized PageRank: teleport to specific nodes instead of uniformly — used in Pinterest's PinSage for personalized recommendations.

Q49. Explain variational autoencoders (VAEs) and their loss function.

VAE: generative model that learns a structured latent space. Encoder: $q_\varphi(z|x) = N(\mu_\varphi(x), \sigma^2_\varphi(x))$. Decoder: $p_\theta(x|z)$. Loss: $ELBO = E[\log p_\theta(x|z)] - KL(q_\varphi(z|x) \,||\, N(0,I))$. First term: reconstruction loss (maximize likelihood of x given z). Second term: KL regularization (keep posterior close to standard Normal prior — forces structured, continuous latent space). Reparameterization trick: $z = \mu + \sigma * \varepsilon$, $\varepsilon \sim N(0,I)$. Makes sampling differentiable — enables backprop through stochastic nodes. Applications: image generation, anomaly detection (high reconstruction loss), drug discovery (smooth interpolation in chemical space), data augmentation. Comparison to GAN: VAEs produce blurry images (optimize MSE-like reconstruction) but have stable training and interpretable latent space. GANs produce sharp images but have training instability.

Q50. What are the key differences between Random Forest and Gradient Boosting?

Random Forest: parallel training (all trees independent). Low variance (averaging decorrelated trees). Naturally handles overfitting. Less sensitive to hyperparameters. Slower at inference (N trees needed). Good for quick baseline. Gradient Boosting: sequential training. Reduces both bias AND variance. More powerful but needs careful tuning (n_estimators, learning_rate, max_depth, subsampling). Prone to overfitting without regularization. Faster at inference (can use early stopping to limit tree count). Usually higher accuracy than RF on tabular data. XGBoost/LightGBM additions to standard GBM: L1/L2 regularization, column subsampling (like RF), approximate split finding, missing value handling, parallel histogram building, GOSS (Gradient-based One-Side Sampling in LightGBM). Rule of thumb: start with LightGBM for tabular data — it's usually best with proper tuning.

Q51. How would you implement a text classification pipeline from raw text to production?

Step 1 — Data: collect and label samples. Check class distribution, handle imbalance. Split temporally if time matters. Step 2 — Preprocessing: lowercase, normalize (unicode), handle HTML/special chars. Step 3 — Baseline: TF-IDF + Logistic Regression. Fast, interpretable, strong baseline. Step 4 — Deep model: fine-tune BERT/RoBERTa. Learning rate 2e-5, batch 32, 3 epochs. Use early stopping. Step 5 — Optimization: if latency is critical, distill to smaller model (DistilBERT), quantize to INT8. Step 6 — Evaluation: classification report (per-class F1). Confusion matrix. Error analysis (look at misclassified examples). Step 7 — Deployment: ONNX export, FastAPI endpoint, load balancing. Cache common predictions. Step 8 — Monitoring: input distribution (embedding drift), prediction distribution, per-class accuracy (alert if any class degrades).

Q52. What is the difference between parametric and non-parametric models and implications for ML?

Parametric: fixed number of parameters regardless of training data size. Examples: linear/logistic regression (d+1 params), NN (architecture defines param count). Fast prediction, compact, can extrapolate. Quality limited by model capacity. Non-parametric: number of parameters grows with data. Examples: KNN (stores all data), kernel SVM (support vectors), Gaussian Process (full covariance matrix). Flexible, can capture complex patterns. Slow prediction, can't extrapolate beyond training range. Semi-parametric: mixture. Example: neural network with KNN retrieval (kNN-LM for language modeling). Modern perspective: transformers with very large context (KV cache for all past tokens) behave somewhat like non-parametric models — memory grows with context length.

Q53. What is conformal prediction and why is it gaining importance?

Conformal prediction: a distribution-free framework for producing prediction intervals/sets with guaranteed coverage. Key property: for any black-box model, conformal prediction produces prediction sets that contain the true label with probability ≥ 1-α, without distributional assumptions. Method: fit model on train set. Calibrate on validation set: compute nonconformity scores (e.g., 1 - P(true class)). At test time: include all classes whose score is below the (1-α) quantile of calibration scores. Why important: ML models are usually overconfident. Medical, legal, financial applications need reliable uncertainty. Conformal prediction gives rigorous coverage guarantees unlike Platt scaling or temperature scaling. Increasingly required in production ML for safety-critical applications.

Q54. How would you train a model to be robust to adversarial examples?

Adversarial examples: small perturbations to inputs (imperceptible to humans) that cause misclassification. FGSM (Fast Gradient Sign Method): $x_adv = x + \varepsilon * \text{sign}(\nabla_x L(x,y))$. Defenses: (1) Adversarial training: augment training set with adversarial examples. Most effective but expensive. PGD adversarial training is the gold standard. (2) Certified defenses: randomized smoothing — add Gaussian noise during inference, provably bounds adversarial perturbation. (3) Feature squeezing: median smoothing, bit depth reduction to remove adversarial perturbations. (4) Detection: separate detector model identifies adversarial inputs. For NLP: character-level perturbations, synonym substitution. BERT-based models are more robust than LSTM to word substitution attacks.

Q55. What are the considerations for deploying ML models on edge devices (mobile, IoT)?

Constraints: limited compute (1-10 GFLOPS), memory (1-4GB RAM), storage (< 100MB model size), battery (energy efficiency), no persistent internet connection. Solutions: (1) Model compression:

quantization to INT8 (4x smaller, faster on hardware INT8 units), pruning, knowledge distillation. (2) Efficient architectures: MobileNet (depthwise separable conv), EfficientNet-Lite, TinyBERT (4 layers). (3) On-device inference: TensorFlow Lite, PyTorch Mobile, ONNX Runtime Mobile. Hardware-specific: Core ML for iPhone, NNAPI for Android. (4) Federated learning: train locally, aggregate updates globally — privacy-preserving, no raw data leaves device. Used by Microsoft (Focused Inbox) and Google (Gboard). (5) Split inference: lightweight feature extraction on device, heavy inference in cloud. Reduces data transmitted.

Q56. Describe the double descent phenomenon.

Classical bias-variance tradeoff predicts: more parameters → more variance → worse generalization. But modern deep learning shows a 'double descent' curve: as model size increases past the interpolation threshold (when training error → 0), test error DECREASES again. Three regimes: (1) Classical (underparameterized): more params → lower bias → lower test error. (2) Near interpolation threshold: models exactly interpolate training data in the most 'complex' way → high test error. (3) Overparameterized: models can fit data in many ways; implicit bias of gradient descent (or of SGD specifically) selects the simplest/flattest solution → good generalization. This explains why modern 175B parameter models generalize without explicit regularization. Benign overfitting hypothesis.

PART 10: BEHAVIORAL — AMAZON LEADERSHIP PRINCIPLES

10. Behavioral & Amazon Leadership Principles

Amazon's Applied Scientist interviews spend 30-40% of time on behavioral questions tied to Leadership Principles. Each interviewer is assigned 2-3 LPs to evaluate. Prepare 2-3 STAR stories per principle.

The 16 Leadership Principles — Key Ones for AS Role

Customer Obsession	Start with customer impact, work backwards. In AS context: always connect your ML work to customer benefit. 'This model reduces latency by 50ms, which improves customer satisfaction by X.'
Are Right A Lot	Good judgment and strong intuition. Back decisions with data. Know when to override data with domain expertise. Show you can make good decisions under uncertainty.
Invent and Simplify	Novel approaches AND removing complexity. Describe a time you found a simpler solution that worked better, or invented a new ML technique for a practical problem.
Dive Deep	Data-driven, connected to details. Interviewers will ask detailed follow-up questions. Know your metrics, hyperparameters, and design decisions cold.
Bias for Action	Calculated risk-taking, speed matters. 'I ran a quick experiment to validate before investing in full development.' Show you don't over-analyze.
Deliver Results	Focus on inputs but also ensure outputs. Quantify your impact. 'This shipped and improved X metric by Y% for Z customers.'

STAR Method — Best Practice

- **Situation (20%):** Brief context. Business problem, team size, timeline.
- **Task (10%):** Your specific responsibility. What were you personally accountable for?

- **Action (50%):** YOUR actions specifically. Use 'I', not 'we'. Detail technical decisions. Show leadership.
- **Result (20%):** Quantified impact. Business metric improved. Cost saved. Users impacted. What did you learn?

Microsoft-Specific: Growth Mindset & Collaboration

Microsoft (under Satya Nadella) evaluates: growth mindset (learn from failure), clarity (communicate complex ML clearly to non-technical stakeholders), energy/energize (inspire the team), and deliver success. Behavioral focus: collaboration, cross-functional influence, learning from failure.

Study Checklist — Prioritized

Highest Priority (Must Know)

- Derive gradients: MSE, cross-entropy, sigmoid — on paper
- Bias-variance: examples of high bias/variance models, fixes
- Attention mechanism: full math, $O(N^2)$ complexity, Flash Attention
- Backpropagation: chain rule, layer-by-layer example
- XGBoost vs Random Forest: second-order gradients, regularization differences
- System design: Recommendation, Fraud Detection, Search (know the full stack)
- A/B testing: design, power analysis, common pitfalls (SRM, novelty effect)

High Priority

- BERT vs GPT vs T5: when to use each, fine-tuning approaches
- LoRA, QLoRA, RLHF: how modern LLMs are fine-tuned
- Causal inference: confounders, diff-in-diff, propensity score
- Feature engineering: encoding, scaling, handling missing data, time series
- Model monitoring: PSI, drift detection, retraining strategies
- LeetCode patterns: sliding window, heap, binary search, graph traversal

Important (May Come Up)

- GNNs, RL basics, VAEs, Diffusion models
- Distributed training: DDP, ZeRO, tensor parallelism
- Model compression: quantization, pruning, distillation
- Optimization theory: LP, convex optimization
- SHAP, LIME, fairness metrics